

Table of Contents

1 SQL Reference — QueryMentor Knowledge Base	4
1.1 Canonical Sample Data	4
2 PART 1: SQL FUNDAMENTALS	6
2.0.1 1.1 What Is SQL	6
2.0.2 1.2 The Five SQL Sublanguages	6
2.0.3 1.3 Case Sensitivity	7
2.0.4 1.4 Relational Model Terms	7
2.0.5 1.5 Keys	7
2.0.6 1.6 Relationship Types	7
2.0.7 1.7 Normalization (1NF–3NF)	8
2.1 1.8 Data Types Across Dialects	8
2.1.1 1.8.1 Numeric Types	8
2.1.2 1.8.2 Text Types	9
2.1.3 1.8.3 Date/Time Types	9
2.1.4 1.8.4 JSON Types	9
2.1.5 1.8.5 Type Conversion	9
2.2 1.9 NULL Semantics	9
2.2.1 1.9.1 What NULL Means	9
2.2.2 1.9.2 Three-Valued Logic	10
2.2.3 1.9.3 Testing for NULL	10
2.2.4 1.9.4 COALESCE and NULLIF	10
2.2.5 1.9.5 NULL in Aggregations	10
2.2.6 1.9.6 NULL in JOINS	10
2.3 1.10 Operators	11
2.3.1 1.10.1 Arithmetic & Comparison	11
2.3.2 1.10.2 Pattern Matching	11
2.3.3 1.10.3 IN, EXISTS, BETWEEN	11
3 PART 2: DDL — DATA DEFINITION LANGUAGE	11
3.0.1 2.1 CREATE TABLE	11
3.0.2 2.2 Temporary Tables	12
3.0.3 2.3 ALTER TABLE	12
3.0.4 2.4 DROP vs TRUNCATE vs DELETE	12
3.0.5 2.5 Constraints	12
3.0.6 2.6 Indexes	13
3.0.7 2.7 Views and Materialized Views	13
3.0.8 2.8 Auto-Increment / Sequences	13
4 PART 3: DML — DATA MANIPULATION LANGUAGE	14
4.0.1 3.1 INSERT	14
4.0.2 3.2 RETURNING (PostgreSQL, SQLite 3.35+)	14
4.0.3 3.3 Upsert	14
4.0.4 3.4 UPDATE	14
4.0.5 3.5 DELETE	15
4.0.6 3.6 Common DML Gotchas	15
5 PART 4: DQL — DATA QUERY LANGUAGE	15
5.0.1 4.1 SELECT Anatomy — Written Order	15
5.0.2 4.2 Execution Order vs Written Order (Critical)	15
5.0.3 4.3 WHERE — Operators Reference	16
5.0.4 4.4 The NOT IN + NULL Gotcha	16

5.1	4.5 JOINS Deep Dive	17
5.1.1	4.5.1 INNER JOIN	17
5.1.2	4.5.2 LEFT JOIN (LEFT OUTER JOIN)	17
5.1.3	4.5.3 RIGHT JOIN	18
5.1.4	4.5.4 FULL OUTER JOIN	18
5.1.5	4.5.5 CROSS JOIN	18
5.1.6	4.5.6 SELF JOIN	19
5.1.7	4.5.7 NATURAL JOIN	19
5.1.8	4.5.8 LATERAL JOIN (PostgreSQL)	19
5.1.9	4.5.9 Join Performance Notes	19
5.2	4.6 Aggregations	20
5.2.1	4.6.1 COUNT Variations	20
5.2.2	4.6.2 SUM, AVG, MIN, MAX	20
5.2.3	4.6.3 GROUP BY	20
5.2.4	4.6.4 HAVING vs WHERE	21
5.2.5	4.6.5 ROLLUP, CUBE, GROUPING SETS	21
5.3	4.7 Subqueries	21
5.3.1	4.7.1 Scalar Subquery	21
5.3.2	4.7.2 Table Subquery (Derived Table)	21
5.3.3	4.7.3 Correlated vs Non-Correlated	22
5.3.4	4.7.4 IN vs EXISTS vs ANY vs ALL	22
5.4	4.8 Set Operations	22
6	PART 5: ADVANCED SQL	23
6.1	5.1 Common Table Expressions (CTEs)	23
6.1.1	5.1.1 Basic WITH Syntax	23
6.1.2	5.1.2 Multiple CTEs	23
6.1.3	5.1.3 CTEs vs Subqueries	23
6.1.4	5.1.4 Recursive CTEs — Syntax	24
6.1.5	5.1.5 Recursive CTE — Org Chart	24
6.1.6	5.1.6 Recursive CTE — Number Series	24
6.1.7	5.1.7 Recursive CTE Gotchas	25
6.2	5.2 Window Functions	25
6.2.1	5.2.1 Why Window Functions?	25
6.2.2	5.2.2 OVER Clause Anatomy	25
6.2.3	5.2.3 Ranking Functions	25
6.2.4	5.2.4 Offset Functions	26
6.2.5	5.2.5 Aggregate Window Functions — Running Totals	26
6.2.6	5.2.6 ROWS vs RANGE	26
6.2.7	5.2.7 Named Windows	26
6.2.8	5.2.8 Window Functions vs GROUP BY	27
6.3	5.3 CASE Expressions	27
6.3.1	5.3.1 Simple vs Searched CASE	27
6.3.2	5.3.2 Pivot With CASE (Rows to Columns)	27
6.4	5.4 NULL Handling Functions	28
6.5	5.5 Date/Time Functions Across Dialects	28
6.6	5.6 String Functions	29
6.7	5.7 JSON Functions	29
6.7.1	5.7.1 PostgreSQL JSONB	29
6.7.2	5.7.2 MySQL JSON	29
6.7.3	5.7.3 SQLite JSON1 Extension	29
6.8	5.8 Full-Text Search	30
7	PART 6: TRANSACTIONS & CONCURRENCY	30

7.1	6.1 ACID Properties	30
7.2	6.2 Transaction Control	30
7.3	6.3 Isolation Levels	31
7.3.1	6.3.1 Read Phenomena	31
7.3.2	6.3.2 Isolation Level Comparison	31
7.3.3	6.3.3 Defaults Per Dialect	31
7.4	6.4 Locking	31
7.5	6.5 Deadlocks	32
7.6	6.6 MVCC (Multi-Version Concurrency Control)	32
8	PART 7: PERFORMANCE & OPTIMIZATION	33
8.1	7.1 Query Execution Plans	33
8.1.1	7.1.1 EXPLAIN vs EXPLAIN ANALYZE	33
8.1.2	7.1.2 Reading a Plan	33
8.1.3	7.1.3 Scan and Join Types	33
8.2	7.2 Indexing Strategies	34
8.3	7.3 Query Optimization Patterns	34
8.4	7.4 Common Anti-Patterns	35
9	PART 8: DIALECT DIFFERENCES APPENDIX	35
9.1	8.1 Data Types	35
9.2	8.2 DDL Syntax	36
9.3	8.3 DML Syntax	36
9.4	8.4 Function Comparisons	37
9.5	8.5 Window Functions & CTEs	37
9.6	8.6 Error Handling	37
10	PART 9: INTERVIEW PREPARATION	37
10.1	9.1 Top 50 SQL Interview Questions	38
10.1.1	9.1.1 Beginner	38
10.1.2	9.1.2 Intermediate	39
10.1.3	9.1.3 Advanced	40
10.2	9.2 Classic Problems With Runnable Solutions	41
10.2.1	9.2.1 Nth Highest Salary — Three Approaches	41
10.2.2	9.2.2 Find Duplicate Records	41
10.2.3	9.2.3 Delete Duplicate Rows, Keeping One	41
10.2.4	9.2.4 Employees Earning More Than Their Manager	42
10.2.5	9.2.5 Running Totals and Moving Averages	42
10.2.6	9.2.6 Find Gaps in a Sequence	42
10.2.7	9.2.7 Pivot Rows to Columns With CASE	42
10.2.8	9.2.8 Hierarchical Query With Recursive CTE	42
10.3	9.3 Query Pattern Cookbook	43
10.3.1	9.3.1 Top N Per Group	43
10.3.2	9.3.2 Year-Over-Year Comparison	43
10.3.3	9.3.3 Month-Over-Month Growth	43
10.3.4	9.3.4 Cohort Analysis (Signup Month)	43
10.3.5	9.3.5 Retention Analysis (% Active N Days Later)	43
10.3.6	9.3.6 Funnel Analysis	43
10.3.7	9.3.7 Sessionization (Gap-Based)	44
10.3.8	9.3.8 Median (50th Percentile)	44
10.3.9	9.3.9 Percentile Calculation (95th)	44
10.3.10	9.3.10 Cumulative Distribution	44
10.3.11	9.3.11 Running Distinct Count	44
10.3.12	9.3.12 Rank Within Partition	44

10.3.19.3.13 First / Last Value in Group	44
10.3.19.3.14 Deduplication (Keep Latest)	45
10.3.19.3.15 Slowly Changing Dimension (Type 2 Lookup)	45
10.3.19.3.16 Date Spine Generation	45
10.3.19.3.17 Moving Average (7-Day)	45
10.3.19.3.18 Recursive Hierarchy Depth	45
10.3.19.3.19 Unpivot (Columns to Rows)	45
10.3.29.3.20 Gap Detection (Missing Dates)	45
10.3.29.3.21 Island Detection (Consecutive Runs)	46
10.3.29.3.22 Consecutive Login Streaks	46
10.3.29.3.23 Cohort Retention Matrix	46
10.3.29.3.24 RFM Analysis (Recency, Frequency, Monetary)	46
10.3.29.3.25 Basket Analysis (Products Bought Together)	46
10.3.29.3.26 Deduplicate and Merge Records	46
10.3.29.3.27 Latest Row Per Group	46
10.3.29.3.28 Top-N With Ties Included	47
10.3.29.3.29 Percent Change Between Rows	47
10.3.39.3.30 Period-Over-Period Comparison	47
11PART 10: CHEAT SHEET	47
11.110.1 DDL Quick Reference	47
11.210.2 DML Quick Reference	47
11.310.3 DQL Quick Reference	47
11.410.4 JOIN Types, One Line Each	48
11.510.5 Aggregate Functions	48
11.610.6 Window Functions	48
11.710.7 Date Functions One-Liners	48
11.810.8 String Functions One-Liners	48
11.910.9 JSON One-Liners (PostgreSQL)	48
11.100.10 Top 15 Dialect Gotchas	49
11.110.11 Top 10 Performance Tips	49
11.120.12 Top 10 Patterns as One-Liners	49

1 SQL Reference — QueryMentor Knowledge Base

Pass 1 of 3. Canonical schema (used throughout, restated inline per section where relevant):

- users(user_id, email, full_name, created_at, country, is_active)
- products(product_id, name, category, price, stock, created_at)
- orders(order_id, user_id, order_date, status, total_amount)
- order_items(item_id, order_id, product_id, quantity, unit_price)
- employees(employee_id, full_name, manager_id, department, salary, hire_date)
- departments(department_id, name, budget)

1.1 Canonical Sample Data

```
CREATE TABLE users (
  user_id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  full_name VARCHAR(100),
  created_at TIMESTAMP DEFAULT NOW(),
```

```

country VARCHAR(2),
is_active BOOLEAN DEFAULT TRUE
);

INSERT INTO users (email, full_name, country, is_active) VALUES
('alice@example.com', 'Alice Chen', 'US', TRUE),
('bob@example.com', 'Bob Diaz', 'UK', TRUE),
('carol@example.com', 'Carol Nguyen', 'CA', FALSE),
('dave@example.com', 'Dave Kim', 'US', TRUE),
('erin@example.com', 'Erin Walsh', 'AU', TRUE),
('frank@example.com', 'Frank Osei', 'US', FALSE),
('gina@example.com', 'Gina Rossi', 'IT', TRUE);

CREATE TABLE products (
product_id SERIAL PRIMARY KEY,
name VARCHAR(200) NOT NULL,
category VARCHAR(50),
price DECIMAL(10,2),
stock INTEGER DEFAULT 0,
created_at TIMESTAMP
);

INSERT INTO products (name, category, price, stock) VALUES
('Wireless Mouse', 'Electronics', 24.99, 150),
('Mechanical Keyboard', 'Electronics', 89.99, 60),
('Standing Desk', 'Furniture', 349.00, 12),
('Desk Lamp', 'Furniture', 29.50, 80),
('Notebook Pack', 'Office', 8.99, 300),
('Noise-Cancel Headphones', 'Electronics', 199.00, 25);

CREATE TABLE orders (
order_id SERIAL PRIMARY KEY,
user_id INTEGER REFERENCES users(user_id),
order_date DATE NOT NULL,
status VARCHAR(20),
total_amount DECIMAL(10,2)
);

INSERT INTO orders (user_id, order_date, status, total_amount) VALUES
(1, '2026-01-05', 'delivered', 114.98),
(1, '2026-02-14', 'shipped', 29.50),
(2, '2026-01-20', 'delivered', 349.00),
(3, '2026-03-01', 'cancelled', 8.99),
(4, '2026-03-10', 'pending', 199.00),
(5, '2026-03-12', 'delivered', 24.99);

CREATE TABLE order_items (
item_id SERIAL PRIMARY KEY,
order_id INTEGER REFERENCES orders(order_id),
product_id INTEGER REFERENCES products(product_id),
quantity INTEGER NOT NULL,
unit_price DECIMAL(10,2)
);

INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES

```

```

(1, 1, 1, 24.99),
(1, 2, 1, 89.99),
(2, 4, 1, 29.50),
(3, 3, 1, 349.00),
(4, 5, 1, 8.99),
(5, 6, 1, 199.00);

CREATE TABLE employees (
  employee_id SERIAL PRIMARY KEY,
  full_name VARCHAR(100),
  manager_id INTEGER REFERENCES employees(employee_id),
  department VARCHAR(50),
  salary DECIMAL(10,2),
  hire_date DATE
);

INSERT INTO employees (full_name, manager_id, department, salary, hire_date) VALUES
('Nora Patel', NULL, 'Engineering', 185000, '2018-03-01'),
('Sam Ortiz', 1, 'Engineering', 142000, '2019-06-15'),
('Liu Wei', 1, 'Engineering', 138000, '2020-01-10'),
('Maya Johnson', NULL, 'Sales', 160000, '2017-09-01'),
('Tom Becker', 4, 'Sales', 95000, '2021-04-20'),
('Priya Rao', 4, 'Sales', 88000, '2022-02-11');

CREATE TABLE departments (
  department_id SERIAL PRIMARY KEY,
  name VARCHAR(100),
  budget DECIMAL(12,2)
);

INSERT INTO departments (name, budget) VALUES
('Engineering', 5000000),
('Sales', 2500000),
('Marketing', 1200000);

```

2 PART 1: SQL FUNDAMENTALS

2.0.1 1.1 What Is SQL

SQL (Structured Query Language) is a declarative language for querying and managing relational databases. You state *what* result you want; the database's query planner decides *how* to get it.

```
-- Declarative: describe the result
SELECT * FROM users WHERE country = 'US';
```

Note: Contrast with imperative code (a for loop checking each row) — SQL never specifies the loop, index, or join order explicitly.

2.0.2 1.2 The Five SQL Sublanguages

Sublanguage	Purpose	Example statements
DDL	Define structure	CREATE, ALTER, DROP, TRUNCATE
DML	Modify data	INSERT, UPDATE, DELETE, MERGE
DQL	Query data	SELECT
DCL	Control access	GRANT, REVOKE
TCL	Manage transactions	BEGIN, COMMIT, ROLLBACK

2.0.3 1.3 Case Sensitivity

- Keywords are case-insensitive: SELECT = select = SeLeCt.
- Convention: uppercase keywords, lowercase identifiers.
- PostgreSQL folds *unquoted* identifiers to lowercase; "Users" (quoted) preserves case.
- MySQL identifier case-sensitivity depends on the OS and lower_case_table_names.
- SQLite is case-insensitive for ASCII identifiers only.

Warning: SELECT * FROM Users; and SELECT * FROM users; may or may not be the same table depending on dialect and OS — don't rely on case to distinguish names.

2.0.4 1.4 Relational Model Terms

Formal term	Common name
Relation	Table
Tuple	Row
Attribute	Column
Cardinality	Number of rows
Degree	Number of columns

2.0.5 1.5 Keys

- Schema:** users(user_id, email, ...)
- Primary key** — uniquely identifies a row, cannot be NULL: user_id.
- Composite primary key** — PK spanning multiple columns, e.g. PRIMARY KEY (order_id, product_id) in a junction table.
- Foreign key** — references another table's PK: orders.user_id REFERENCES users(user_id).
- Surrogate key** — artificial, auto-incrementing (user_id SERIAL).
- Natural key** — real-world value that could identify a row (email).

-- Schema: orders(order_id PK, user_id FK -> users)

```
CREATE TABLE orders (
  order_id SERIAL PRIMARY KEY,
  user_id INTEGER REFERENCES users(user_id)
);
```

Best Practice: Prefer surrogate keys for PKs; use natural keys (like email) as UNIQUE constraints instead.

2.0.6 1.6 Relationship Types

- One-to-one:** users ↔ a hypothetical user_profiles(user_id PK/FK).
- One-to-many:** users(user_id) → orders(user_id FK) — one user, many orders.

- **Many-to-many:** needs a junction table, e.g. order_items(order_id FK, product_id FK) links orders and products.

-- Schema: order_items(order_id FK -> orders, product_id FK -> products)

```
SELECT o.order_id, p.name
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id;
```

2.0.7 1.7 Normalization (1NF-3NF)

- **1NF:** atomic values, no repeating groups. Violation: phone_numbers = '555-1111,555-2222' in one column.
- **2NF:** 1NF + no partial dependency on part of a composite key.
- **3NF:** 2NF + no transitive dependency (non-key column depending on another non-key column).

-- Violates 1NF

-- users(user_id, phones TEXT) -- '555-1111,555-2222'

-- Fixed: separate table

-- Schema: user_phones(user_id FK -> users, phone)

```
CREATE TABLE user_phones (
  user_id INTEGER REFERENCES users(user_id),
  phone VARCHAR(20)
);
```

Anti-Pattern: Comma-separated values in a column. Querying “does user 3 have phone X” becomes a string search instead of an indexed lookup.

2.1 1.8 Data Types Across Dialects

2.1.1 1.8.1 Numeric Types

Type	PostgreSQL	MySQL	SQLite
Small int	SMALLINT	SMALLINT	INTEGER
Int	INTEGER	INT	INTEGER
Big int	BIGINT	BIGINT	INTEGER
Auto-increment	SERIAL	AUTO_INCREMENT	AUTOINCREMENT
Exact decimal	NUMERIC(p,s)	DECIMAL(p,s)	NUMERIC
Float	REAL, DOUBLE PRECISION	FLOAT, DOUBLE	REAL
Boolean	BOOLEAN	TINYINT(1)	INTEGER (0/1)

-- Schema: products(price DECIMAL(10,2))

-- Money must use exact decimal types, never FLOAT

```
SELECT price * 1.08 AS price_with_tax FROM products WHERE product_id = 1;
```

```
price_with_tax
26.99
```

Warning: Never use FLOAT/REAL for money — binary floating point can’t represent 0.10 exactly, causing rounding errors after repeated arithmetic.

2.1.2 1.8.2 Text Types

Type	PostgreSQL	MySQL	SQLite
Fixed-length	CHAR(n)	CHAR(n)	TEXT
Variable-length	VARCHAR(n)	VARCHAR(n)	TEXT
Unlimited	TEXT	TEXT/LONGTEXT	TEXT

- CHAR(n) pads with trailing spaces; VARCHAR(n) does not.
- SQLite has one universal TEXT type regardless of what you declare (dynamic typing).

2.1.3 1.8.3 Date/Time Types

Type	PostgreSQL	MySQL	SQLite
Date	DATE	DATE	TEXT (ISO 8601)
Timestamp	TIMESTAMP	DATETIME	TEXT
Timestamp + TZ	TIMESTAMPTZ	TIMESTAMP (session TZ)	not native

Best Practice: In PostgreSQL, default to TIMESTAMPTZ over TIMESTAMP — it stores an absolute instant instead of an ambiguous local wall-clock value.

2.1.4 1.8.4 JSON Types

- PostgreSQL: JSON (text, unindexed) vs JSONB (binary, indexable) — prefer JSONB almost always.
- MySQL: JSON (5.7+ [verify]).
- SQLite: no native JSON column type; JSON functions via the json1 extension operate on TEXT.

2.1.5 1.8.5 Type Conversion

```
-- Schema: orders(total_amount DECIMAL)
SELECT CAST(total_amount AS INTEGER) AS rounded_amount
FROM orders WHERE order_id = 1;
-- PostgreSQL shorthand: total_amount::INTEGER
```

rounded_amount
115

Note: MySQL uses CAST(x AS SIGNED) / CAST(x AS UNSIGNED) for integers rather than CAST(x AS INTEGER).

2.2 1.9 NULL Semantics

2.2.1 1.9.1 What NULL Means

NULL represents *unknown*, not zero or empty string. It propagates: NULL + 1 is NULL, and NULL = NULL evaluates to UNKNOWN, not TRUE.

2.2.2 1.9.2 Three-Valued Logic

A	B	A AND B	A OR B
TRUE	NULL	NULL	TRUE
FALSE	NULL	FALSE	NULL
NULL	NULL	NULL	NULL

2.2.3 1.9.3 Testing for NULL

```
-- Schema: users(is_active BOOLEAN, full_name)
SELECT full_name FROM users WHERE full_name IS NULL;
-- WRONG – never returns rows even if full_name is NULL:
-- SELECT full_name FROM users WHERE full_name = NULL;
```

Warning: column = NULL always evaluates to UNKNOWN (treated as false in WHERE).
Always use IS NULL / IS NOT NULL.

2.2.4 1.9.4 COALESCE and NULLIF

```
-- Schema: users(full_name)
SELECT COALESCE(full_name, 'Unknown User') AS display_name
FROM users WHERE user_id = 3;
```

```
SELECT NULLIF(country, 'US') AS non_us_country
FROM users WHERE user_id = 1;
```

display_name
Carol Nguyen

NULLIF(a, b) returns NULL if a = b, otherwise returns a — commonly used to avoid divide-by-zero: total / NULLIF(count, 0).

2.2.5 1.9.5 NULL in Aggregations

```
-- Schema: orders(total_amount)
SELECT COUNT(*) AS all_rows, COUNT(total_amount) AS non_null_amounts
FROM orders WHERE status = 'cancelled';
```

COUNT(*) counts every row; COUNT(column) skips NULLs; SUM/AVG/MIN/MAX all skip NULLs and return NULL if every input was NULL.

2.2.6 1.9.6 NULL in JOINS

NULL never matches NULL in a join condition — a.x = b.x is UNKNOWN when either side is NULL, so the row is excluded just like FALSE.

2.3 1.10 Operators

2.3.1 1.10.1 Arithmetic & Comparison

```
-- Schema: order_items(quantity, unit_price)
SELECT quantity * unit_price AS line_total FROM order_items WHERE item_id = 1;
```

- Standard: + - * / %. PostgreSQL adds ^ (power); MySQL has DIV for integer division.
- Division by zero: PostgreSQL raises an error; MySQL returns NULL.
- Use <> over != for ANSI portability (both work in PG/MySQL/SQLite, but <> is the standard).

2.3.2 1.10.2 Pattern Matching

```
-- Schema: users(email)
SELECT email FROM users WHERE email LIKE 'a%';
```

email
alice@example.com

- % = zero or more characters, _ = exactly one character.
- LIKE is case-sensitive in PostgreSQL, case-insensitive by default in MySQL.
- ILIKE (PostgreSQL only) is case-insensitive.

Warning: LIKE '%foo' (leading wildcard) cannot use a standard B-tree index — it forces a full scan.

2.3.3 1.10.3 IN, EXISTS, BETWEEN

```
-- Schema: orders(status, order_date)
SELECT * FROM orders WHERE status IN ('pending', 'shipped');
SELECT * FROM orders WHERE order_date BETWEEN '2026-01-01' AND '2026-01-31';
```

BETWEEN is inclusive on both endpoints.

Warning: NOT IN (subquery) returns **zero rows** if the subquery produces even one NULL — because $x \neq \text{NULL}$ is UNKNOWN for every comparison. Use NOT EXISTS instead.

3 PART 2: DDL — DATA DEFINITION LANGUAGE

3.0.1 2.1 CREATE TABLE

```
CREATE TABLE users (
  user_id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  full_name VARCHAR(100),
  created_at TIMESTAMP DEFAULT NOW(),
  country VARCHAR(2),
  is_active BOOLEAN DEFAULT TRUE,
  CHECK (country ~ '^[A-Z]{2}$')
);
```

CREATE TABLE IF NOT EXISTS users (...) is supported in PostgreSQL, MySQL, and SQLite — useful for idempotent migration scripts.

3.0.2 2.2 Temporary Tables

```
-- PostgreSQL / SQLite: session or connection scoped
CREATE TEMP TABLE recent_orders AS
SELECT * FROM orders WHERE order_date > '2026-03-01';
-- MySQL:
CREATE TEMPORARY TABLE recent_orders AS
SELECT * FROM orders WHERE order_date > '2026-03-01';
```

Note: CREATE TABLE ... AS SELECT copies data but **not** constraints or indexes from the source table.

3.0.3 2.3 ALTER TABLE

```
-- Schema: users(...)
ALTER TABLE users ADD COLUMN phone VARCHAR(20);
ALTER TABLE users DROP COLUMN phone;
ALTER TABLE users RENAME COLUMN full_name TO display_name; -- PG, MySQL 8+, SQLite 3.25+
```

Operation	PostgreSQL	MySQL	SQLite
Modify type	ALTER COLUMN c TYPE t	MODIFY c t	not supported directly
Rename table	RENAME TO	RENAME TABLE t TO t2	RENAME TO

Warning: SQLite cannot change a column's type in place — the standard workaround is to create a new table, copy data, drop the old table, and rename.

3.0.4 2.4 DROP vs TRUNCATE vs DELETE

Aspect	DELETE	TRUNCATE	DROP
Removes	Rows (optionally filtered)	All rows	Rows + table structure
WHERE clause	Yes	No	No
Triggers fire	Yes	No	No
Rollback-able	Yes	Yes (PG), No (MySQL)	Yes

```
DELETE FROM orders WHERE status = 'cancelled';
TRUNCATE TABLE orders RESTART IDENTITY; -- Postgres: also resets the sequence
DROP TABLE IF EXISTS orders CASCADE;    -- Postgres: also drops dependent objects
```

3.0.5 2.5 Constraints

```
-- Schema: order_items(order_id FK, product_id FK, quantity)
ALTER TABLE order_items
  ADD CONSTRAINT fk_order FOREIGN KEY (order_id) REFERENCES orders(order_id)
  ON DELETE CASCADE,
  ADD CONSTRAINT qty_positive CHECK (quantity > 0);
```

ON DELETE options: NO ACTION (default, errors), CASCADE (delete children), SET NULL, SET DEFAULT, RESTRICT.

Best Practice: Always index foreign-key columns — they're not automatically indexed in PostgreSQL or SQLite (MySQL InnoDB does index them automatically).

3.0.6 2.6 Indexes

```
-- Schema: users(email)
CREATE INDEX idx_users_email ON users(email);
CREATE UNIQUE INDEX idx_users_email_unique ON users(email);
CREATE INDEX idx_active_users ON users(email) WHERE is_active = TRUE; -- partial, PG only
```

Index type	Best for
B-tree (default)	=, <, >, BETWEEN, prefix LIKE 'x%'
Hash	equality only
GIN (PG)	JSONB, arrays, full-text
GiST (PG)	geometric, range types
BRIN (PG)	huge, naturally-sorted tables

Anti-Pattern: Indexing a low-cardinality boolean column alone — the planner usually prefers a sequential scan anyway.

3.0.7 2.7 Views and Materialized Views

```
-- Schema: orders(user_id, total_amount, status)
CREATE VIEW active_order_totals AS
SELECT user_id, SUM(total_amount) AS lifetime_spend
FROM orders WHERE status <> 'cancelled'
GROUP BY user_id;

-- Materialized (PostgreSQL only; no native MySQL equivalent)
CREATE MATERIALIZED VIEW mv_order_totals AS
SELECT user_id, SUM(total_amount) AS lifetime_spend FROM orders GROUP BY user_id;
REFRESH MATERIALIZED VIEW mv_order_totals;
```

A regular view re-runs its query every time; a materialized view stores results and must be explicitly refreshed.

3.0.8 2.8 Auto-Increment / Sequences

Feature	PostgreSQL	MySQL	SQLite
Keyword	SERIAL / GENERATED ALWAYS AS IDENTITY	AUTO_INCREMENT	AUTOINCREMENT
Fetch last ID	RETURNING clause	LAST_INSERT_ID()	last_insert_rowid()
Gap-free	No	No	No

```
CREATE TABLE users_v2 (
  user_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email VARCHAR(255)
);
```

Note: SERIAL still works in PostgreSQL but GENERATED ALWAYS AS IDENTITY is the modern, SQL-standard equivalent.

4 PART 3: DML — DATA MANIPULATION LANGUAGE

4.0.1 3.1 INSERT

```
-- Schema: users(email, full_name, country)
INSERT INTO users (email, full_name, country) VALUES
  ('hank@example.com', 'Hank Ford', 'US'),
  ('ivy@example.com', 'Ivy Zhou', 'CN');

-- INSERT ... SELECT
-- Schema: users(user_id, email, is_active)
INSERT INTO users_archive (user_id, email)
SELECT user_id, email FROM users WHERE is_active = FALSE;
```

4.0.2 3.2 RETURNING (PostgreSQL, SQLite 3.35+)

```
-- Schema: users(email)
INSERT INTO users (email) VALUES ('jack@example.com') RETURNING user_id, created_at;
```

user_id	created_at
8	2026-03-15 10:02:00

MySQL has no RETURNING; use LAST_INSERT_ID() for auto-generated IDs after INSERT.

4.0.3 3.3 Upsert

```
-- Schema: users(email UNIQUE, full_name)
-- PostgreSQL / SQLite:
INSERT INTO users (email, full_name) VALUES ('alice@example.com', 'Alice C.')
ON CONFLICT (email) DO UPDATE SET full_name = EXCLUDED.full_name;

-- MySQL:
INSERT INTO users (email, full_name) VALUES ('alice@example.com', 'Alice C.')
ON DUPLICATE KEY UPDATE full_name = VALUES(full_name);
```

4.0.4 3.4 UPDATE

```
-- Schema: users(is_active, user_id)
UPDATE users SET is_active = FALSE WHERE user_id = 3;

-- UPDATE with a join – Schema: orders(user_id, status), users(user_id, country)
-- PostgreSQL:
UPDATE orders o SET status = 'priority'
FROM users u WHERE o.user_id = u.user_id AND u.country = 'US';

-- MySQL:
UPDATE orders o JOIN users u ON o.user_id = u.user_id
SET o.status = 'priority' WHERE u.country = 'US';
```

Warning: An UPDATE with no WHERE clause updates every row in the table. Always double-check the filter before running in production.

4.0.5 3.5 DELETE

```
-- Schema: orders(user_id), users(user_id, is_active)
-- PostgreSQL:
DELETE FROM orders o USING users u
WHERE o.user_id = u.user_id AND u.is_active = FALSE;

-- MySQL:
DELETE o FROM orders o JOIN users u ON o.user_id = u.user_id
WHERE u.is_active = FALSE;

-- Schema: orders(status, order_id)
DELETE FROM orders WHERE status = 'cancelled' RETURNING order_id; -- PG/SQLite
```

4.0.6 3.6 Common DML Gotchas

Anti-Pattern: DELETE FROM orders WHERE user_id IN (SELECT user_id FROM users WHERE is_active = FALSE) — correct, but if the subquery could ever return NULL values mixed with real ones, prefer EXISTS for safety and clarity.

Best Practice: Wrap multi-statement UPDATE/DELETE sequences in an explicit transaction (BEGIN ... COMMIT) so a mistake can be rolled back before it's permanent.

5 PART 4: DQL — DATA QUERY LANGUAGE

Canonical schema: users(user_id, email, full_name, created_at, country, is_active) · products(product_id, name, category, price, stock, created_at) · orders(order_id, user_id, order_date, status, total_amount) · order_items(item_id, order_id, product_id, quantity, unit_price) · employees(employee_id, full_name, manager_id, department, salary, hire_date) · departments(department_id, name, budget)

5.0.1 4.1 SELECT Anatomy — Written Order

```
SELECT [DISTINCT] column_list
FROM table_list
[JOIN ...]
WHERE condition
GROUP BY column_list
HAVING condition
ORDER BY column_list [ASC|DESC]
LIMIT n [OFFSET m];
```

Each clause is optional except SELECT; the order above is the only order the parser accepts, regardless of the order the engine actually evaluates them in (see 4.2).

5.0.2 4.2 Execution Order vs Written Order (Critical)

```
-- Schema: orders(user_id, status, total_amount)
SELECT user_id, SUM(total_amount) AS total
FROM orders
WHERE status <> 'cancelled'
GROUP BY user_id
```

```
HAVING SUM(total_amount) > 100
ORDER BY total DESC
LIMIT 5;
```

Execution order — not the order it's typed:

1. **FROM** — start with all rows of orders.
2. **WHERE** — drop rows where status = 'cancelled', evaluated per row, before any grouping.
3. **GROUP BY** — collapse remaining rows into one group per user_id.
4. **HAVING** — keep only groups whose SUM(total_amount) > 100.
5. **SELECT** — project user_id and the computed total.
6. **DISTINCT** (if present) — deduplicate the projected rows.
7. **ORDER BY** — sort by total.
8. **LIMIT** — slice the top 5.

Note: Because SELECT runs *after* WHERE, a column alias defined in SELECT (like total) cannot be referenced in WHERE — the alias doesn't exist yet at that stage. It usually *can* be used in ORDER BY and GROUP BY (PostgreSQL, MySQL, SQLite all allow this) because those run at or after the SELECT step.

5.0.3 4.3 WHERE — Operators Reference

```
-- Schema: orders(status, total_amount, order_date)
SELECT * FROM orders
WHERE status IN ('pending', 'shipped')
      AND total_amount BETWEEN 20 AND 400
      AND order_date >= '2026-01-01';
```

Category	Operators
Comparison	= <> < > <= >=
Logical	AND OR NOT (precedence: NOT > AND > OR)
Membership	IN, NOT IN
Range	BETWEEN a AND b (inclusive both ends)
Pattern	LIKE, ILIKE (PG only), NOT LIKE
Null test	IS NULL, IS NOT NULL
Existence	EXISTS, NOT EXISTS
Regex	~ ~* !~ !~* (PG), REGEXP (MySQL/SQLite with extension)

Best Practice: Always use <> instead of != for ANSI portability, though both work in all three dialects covered here.

5.0.4 4.4 The NOT IN + NULL Gotcha

```
-- Schema: orders(user_id) may contain a NULL user_id; users(user_id)
-- Returns ZERO rows if even one orders.user_id is NULL — even users with no orders at all
SELECT * FROM users u
WHERE u.user_id NOT IN (SELECT user_id FROM orders);
```

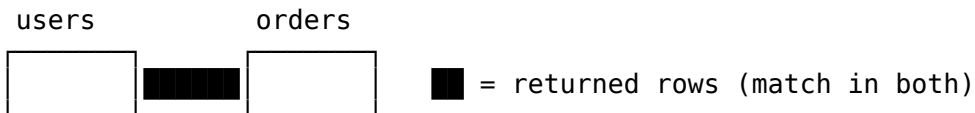
NOT IN expands to user_id <> v1 AND user_id <> v2 AND ... AND user_id <> NULL. Since x <> NULL is UNKNOWN, the whole AND chain becomes UNKNOWN, and WHERE discards any row whose condition isn't TRUE — silently returning nothing.

```
-- Fix: NOT EXISTS is NULL-safe
SELECT * FROM users u
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.user_id);
```

Warning: This is one of the most common silent-bug patterns in production SQL. Prefer `NOT EXISTS` over `NOT IN` whenever the subquery column could ever contain `NULL`.

5.1 4.5 JOINS Deep Dive

5.1.1 4.5.1 INNER JOIN

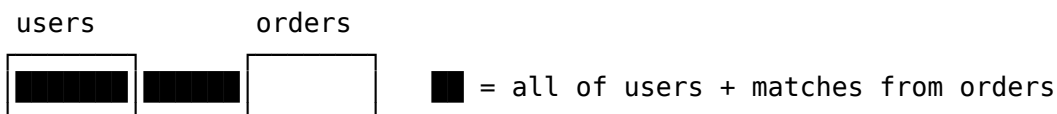


```
-- Schema: users(user_id, full_name), orders(user_id, total_amount)
SELECT u.full_name, o.total_amount
FROM users u
INNER JOIN orders o ON u.user_id = o.user_id;
```

full_name	total_amount
Alice Chen	114.98
Alice Chen	29.50
Bob Diaz	349.00
Carol Nguyen	8.99
Dave Kim	199.00
Erin Walsh	24.99

Only rows with a match in **both** tables appear. Users Frank and Gina, who have no orders, are excluded entirely.

5.1.2 4.5.2 LEFT JOIN (LEFT OUTER JOIN)



```
-- Schema: users(user_id, full_name), orders(user_id, total_amount)
SELECT u.full_name, o.total_amount
FROM users u
LEFT JOIN orders o ON u.user_id = o.user_id;
```

full_name	total_amount
Alice Chen	114.98
Alice Chen	29.50
Bob Diaz	349.00
Carol Nguyen	8.99
Dave Kim	199.00
Erin Walsh	24.99
Frank Osei	NULL
Gina Rossi	NULL

Anti-Pattern: LEFT JOIN orders o ON u.user_id = o.user_id WHERE o.status = 'delivered' silently turns into an INNER JOIN — the WHERE clause runs after the join and discards the NULL-filled rows for users with no matching order, since NULL = 'delivered' is UNKNOWN.

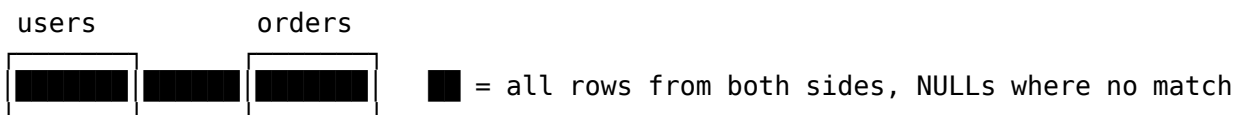
Recommended: Put the extra condition in the ON clause instead: LEFT JOIN orders o ON u.user_id = o.user_id AND o.status = 'delivered' — this preserves every user row while still filtering which orders attach to them.

5.1.3 4.5.3 RIGHT JOIN

```
-- Schema: orders(user_id, total_amount), users(user_id, full_name)
SELECT u.full_name, o.total_amount
FROM orders o
RIGHT JOIN users u ON o.user_id = u.user_id;
```

Identical result to the LEFT JOIN example above, with table order swapped. RIGHT JOIN is rarely used in practice because any RIGHT JOIN can be rewritten as a LEFT JOIN by swapping the FROM and joined table — most style guides prefer that for consistency.

5.1.4 4.5.4 FULL OUTER JOIN



```
-- PostgreSQL / SQLite 3.39+ [verify]:
SELECT u.full_name, o.order_id
FROM users u
FULL OUTER JOIN orders o ON u.user_id = o.user_id;
```

```
-- MySQL has no FULL OUTER JOIN — emulate with UNION:
SELECT u.full_name, o.order_id FROM users u LEFT JOIN orders o ON u.user_id = o.user_id
UNION
SELECT u.full_name, o.order_id FROM users u RIGHT JOIN orders o ON u.user_id = o.user_id;
```

Returns every row from both tables, with NULL filled in on whichever side has no match.

5.1.5 4.5.5 CROSS JOIN

```
-- Schema: users(user_id), products(product_id)
SELECT COUNT(*) AS combinations FROM users CROSS JOIN products;
```

combinations
42

A CROSS JOIN produces the Cartesian product — every row of the left table paired with every row of the right (7 users × 6 products = 42). Useful for generating combinations or date spines; dangerous by accident (1,000 × 1,000 = 1,000,000 rows).

5.1.6 4.5.6 SELF JOIN

```
-- Schema: employees(employee_id, full_name, manager_id)
SELECT e.full_name AS employee, m.full_name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

employee	manager
Nora Patel	NULL
Sam Ortiz	Nora Patel
Liu Wei	Nora Patel
Maya Johnson	NULL
Tom Becker	Maya Johnson
Priya Rao	Maya Johnson

A **self join** joins a table to itself using two different aliases — the classic use case is resolving a hierarchical foreign key like `manager_id`.

5.1.7 4.5.7 NATURAL JOIN

```
-- Schema: orders(user_id, ...), users(user_id, ...)
SELECT * FROM orders NATURAL JOIN users;
```

NATURAL JOIN implicitly joins on every column with a matching name in both tables.

Anti-Pattern: Avoid NATURAL JOIN in production code. If someone later adds an unrelated column with the same name to either table (e.g. both tables gain a `created_at`), the join silently changes behavior with no error — a fragile trap that explicit ON conditions avoid entirely.

5.1.8 4.5.8 LATERAL JOIN (PostgreSQL)

```
-- Schema: users(user_id, full_name), orders(user_id, order_date, total_amount)
SELECT u.full_name, recent.order_date, recent.total_amount
FROM users u
CROSS JOIN LATERAL (
  SELECT order_date, total_amount
  FROM orders o
  WHERE o.user_id = u.user_id
  ORDER BY order_date DESC
  LIMIT 1
) recent;
```

LATERAL lets the subquery reference columns from tables that precede it in the FROM list — impossible with an ordinary subquery. The canonical use case is “top N rows per group,” here fetching each user’s single most recent order.

5.1.9 4.5.9 Join Performance Notes

Best Practice: Always join on indexed columns — an unindexed join column forces a sequential scan on that side regardless of join algorithm. The planner picks between **nested loop** (small outer set), **hash join** (build a hash table on the smaller side), and **merge join** (both inputs pre-sorted) based on table sizes and available indexes — see Part 7 for reading the actual plan.

5.2 4.6 Aggregations

5.2.1 4.6.1 COUNT Variations

```
-- Schema: orders(order_id, status, user_id)
SELECT
  COUNT(*)           AS all_rows,
  COUNT(status)      AS non_null_status,
  COUNT(DISTINCT user_id) AS unique_users
FROM orders;
```

all_rows	non_null_status	unique_users
6	6	5

COUNT(*) counts every row regardless of NULLs; COUNT(column) counts only non-NULL values in that column; COUNT(DISTINCT column) counts distinct non-NULL values (here, 5 unique users even though there are 6 orders, since one user has two orders).

Note: COUNT(1) behaves identically to COUNT(*) in every modern optimizer — the belief that it's faster is a myth.

5.2.2 4.6.2 SUM, AVG, MIN, MAX

```
-- Schema: orders(total_amount)
SELECT SUM(total_amount) AS revenue, AVG(total_amount) AS avg_order,
       MIN(total_amount) AS smallest, MAX(total_amount) AS largest
FROM orders;
```

All four skip NULL values; if every input is NULL, SUM/AVG/MIN/MAX return NULL (not zero).

5.2.3 4.6.3 GROUP BY

```
-- Schema: orders(status, total_amount)
SELECT status, COUNT(*) AS order_count, SUM(total_amount) AS total
FROM orders
GROUP BY status;
```

status	order_count	total
delivered	3	488.97
shipped	1	29.50
cancelled	1	8.99
pending	1	199.00

Warning: Every non-aggregated column in SELECT must appear in GROUP BY (PostgreSQL and SQLite enforce this strictly; MySQL allows it under ONLY_FULL_GROUP_BY being disabled, but the result is then non-deterministic — don't rely on it).

5.2.4 4.6.4 HAVING vs WHERE

```
-- Schema: orders(user_id, status, total_amount)
-- WHERE filters individual rows BEFORE grouping; HAVING filters GROUPS after
SELECT user_id, SUM(total_amount) AS total
FROM orders
WHERE status <> 'cancelled'
GROUP BY user_id
HAVING SUM(total_amount) > 100;
```

Aspect	WHERE	HAVING
Filters	Rows	Groups
Runs	Before GROUP BY	After GROUP BY
Aggregates allowed	No	Yes

5.2.5 4.6.5 ROLLUP, CUBE, GROUPING SETS

```
-- Schema: orders(status, total_amount) – PostgreSQL, MySQL 8.0+ [verify]; not in SQLite
SELECT status, SUM(total_amount) AS total, GROUPING(status) AS is_subtotal
FROM orders
GROUP BY ROLLUP(status);
```

status	total	is_subtotal
cancelled	8.99	0
delivered	488.97	0
pending	199.00	0
shipped	29.50	0
NULL	726.46	1

ROLLUP adds hierarchical subtotal rows (here just a grand total, since there's one grouping column); CUBE adds subtotals for every combination of grouping columns; GROUPING SETS lets you specify the exact combinations explicitly. GROUPING(col) returns 1 when the NULL in that row represents a subtotal rather than a real data value.

5.3 4.7 Subqueries

5.3.1 4.7.1 Scalar Subquery

```
-- Schema: products(name, price)
SELECT name, price, (SELECT AVG(price) FROM products) AS avg_price
FROM products;
```

A scalar subquery returns exactly one row and one column — usable anywhere a single value is expected.

5.3.2 4.7.2 Table Subquery (Derived Table)

```
-- Schema: orders(user_id, total_amount)
SELECT sub.user_id, sub.order_count
FROM (
```

```

SELECT user_id, COUNT(*) AS order_count
FROM orders
GROUP BY user_id
) AS sub
WHERE sub.order_count > 1;

```

A subquery in FROM must have an alias in PostgreSQL and SQLite (MySQL also requires one since 8.0).

5.3.3 4.7.3 Correlated vs Non-Correlated

```

-- Correlated: references the outer row (u.user_id), runs conceptually once per outer row
-- Schema: users(user_id), orders(user_id, status)

```

```

SELECT full_name FROM users u
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.user_id = u.user_id AND o.status = 'delivered'
);

```

```

-- Non-correlated: independent of the outer query, evaluated once

```

```

SELECT full_name FROM users
WHERE user_id IN (SELECT user_id FROM orders WHERE status = 'delivered');

```

5.3.4 4.7.4 IN vs EXISTS vs ANY vs ALL

Construct	Meaning	Performance note
IN (subquery)	Value matches any row in result set	May materialize the full subquery result
EXISTS (subquery)	Subquery returns at least one row	Short-circuits at first match — usually faster for large sets
= ANY (subquery)	Equivalent to IN	Same as IN
> ALL (subquery)	Greater than every value returned	Errors/empty differently than MAX() comparison on empty sets

Best Practice: For large subqueries, prefer EXISTS/NOT EXISTS over IN/NOT IN — both for performance and to sidestep the NOT IN + NULL gotcha in 4.4.

5.4 4.8 Set Operations

```

-- Schema: users(country) – assume a second table eu_users(country) for illustration

```

```

SELECT country FROM users WHERE country = 'US'
UNION

```

```

SELECT country FROM users WHERE is_active = TRUE; -- UNION removes duplicate rows

```

```

SELECT country FROM users WHERE country = 'US'
UNION ALL

```

```

SELECT country FROM users WHERE is_active = TRUE; -- UNION ALL keeps duplicates, faster

```

```

SELECT user_id FROM users

```

```

INTERSECT

```

```

SELECT user_id FROM orders;

```

```

-- PG, MySQL 8.0.31+ [verify], SQLite

```

```
SELECT user_id FROM users
EXCEPT
SELECT user_id FROM orders;
```

-- PG/SQL standard; MySQL 8.0.31+ [ve

Best Practice: Default to UNION ALL unless you specifically need deduplication — UNION performs an implicit sort/hash to remove duplicates, which costs more on large result sets. All set operations require both queries to return the same number of columns with compatible types.

6 PART 5: ADVANCED SQL

Canonical schema: users(user_id, email, full_name, created_at, country, is_active) · products(product_id, name, category, price, stock, created_at) · orders(order_id, user_id, order_date, status, total_amount) · order_items(item_id, order_id, product_id, quantity, unit_price) · employees(employee_id, full_name, manager_id, department, salary, hire_date) · departments(department_id, name, budget)

6.1 5.1 Common Table Expressions (CTEs)

6.1.1 5.1.1 Basic WITH Syntax

```
-- Schema: orders(user_id, total_amount, status)
WITH active_orders AS (
  SELECT * FROM orders WHERE status <> 'cancelled'
)
SELECT user_id, SUM(total_amount) AS total
FROM active_orders
GROUP BY user_id;
```

A CTE is a named, temporary result set scoped to the single statement that follows it.

6.1.2 5.1.2 Multiple CTEs

```
-- Schema: users(user_id, is_active), orders(user_id, order_date, total_amount)
WITH
  active_users AS (SELECT user_id FROM users WHERE is_active = TRUE),
  recent_orders AS (SELECT * FROM orders WHERE order_date > '2026-02-01')
SELECT au.user_id, ro.total_amount
FROM active_users au
JOIN recent_orders ro ON au.user_id = ro.user_id;
```

Later CTEs can reference earlier ones in the same WITH block.

6.1.3 5.1.3 CTEs vs Subqueries

- **Readability:** a CTE names the intermediate result, making the query read top-to-bottom instead of nesting inward.
- **Reuse:** a CTE can be referenced multiple times in the outer query without repeating its SQL.
- **PostgreSQL 12+ [verify]:** CTEs are inlined (treated like subqueries for optimization) by default; add MATERIALIZED to force PostgreSQL to compute it once and cache the result, useful when the CTE is expensive and referenced repeatedly.

```
WITH expensive AS MATERIALIZED (
  SELECT user_id, COUNT(*) FROM orders GROUP BY user_id
)
SELECT * FROM expensive WHERE count > 1;
```

Note: Before Postgres 12, CTEs were always an “optimization fence” — never in-lined, always materialized — which sometimes hurt performance compared to an equivalent subquery.

6.1.4 5.1.4 Recursive CTEs — Syntax

```
WITH RECURSIVE cte_name AS (
  -- anchor member (runs once)
  SELECT ...
  UNION ALL
  -- recursive member (runs repeatedly, referencing cte_name)
  SELECT ... FROM cte_name JOIN ...
)
SELECT * FROM cte_name;
```

6.1.5 5.1.5 Recursive CTE — Org Chart

```
-- Schema: employees(employee_id, full_name, manager_id)
WITH RECURSIVE org_chart AS (
  SELECT employee_id, full_name, manager_id, 1 AS level
  FROM employees WHERE manager_id IS NULL -- anchor: top-level managers
  UNION ALL
  SELECT e.employee_id, e.full_name, e.manager_id, oc.level + 1
  FROM employees e
  JOIN org_chart oc ON e.manager_id = oc.employee_id
)
SELECT full_name, level FROM org_chart ORDER BY level, full_name;
```

full_name	level
Maya Johnson	1
Nora Patel	1
Liu Wei	2
Sam Ortiz	2
Priya Rao	2
Tom Becker	2

6.1.6 5.1.6 Recursive CTE — Number Series

```
WITH RECURSIVE numbers AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM numbers WHERE n < 10
)
SELECT n FROM numbers;
```

Generates 1 through 10 — a common pattern for building date spines or filling gaps.

6.1.7 5.1.7 Recursive CTE Gotchas

Warning: A recursive CTE with no terminating condition on the recursive member loops forever. Always ensure the recursive branch has a WHERE clause (or equivalent) that eventually stops matching.

- PostgreSQL: add a CYCLE clause to detect and stop infinite cycles in graph-like data: ... UNION ALL SELECT ... CYCLE employee_id SET is_cycle USING path.
- MySQL: cte_max_recursion_depth (default 1000 [verify]) aborts runaway recursion with an error.
- **Best Practice:** Set a statement timeout as a safety net whenever writing new recursive CTEs against unfamiliar data.

6.2 5.2 Window Functions

6.2.1 5.2.1 Why Window Functions?

-- GROUP BY collapses to one row per user – loses the individual order rows
SELECT user_id, COUNT(*) **FROM** orders **GROUP BY** user_id;

-- A window function keeps every row AND adds the aggregate as a new column
-- Schema: orders(order_id, user_id, total_amount)

SELECT order_id, user_id, total_amount,
COUNT(*) **OVER** (**PARTITION BY** user_id) **AS** user_order_count
FROM orders;

order_id	user_id	total_amount	user_order_count
1	1	114.98	2
2	1	29.50	2
3	2	349.00	1

6.2.2 5.2.2 OVER Clause Anatomy

```
function_name(args) OVER (  
  [PARTITION BY col1, col2]  
  [ORDER BY col3 [ASC|DESC]]  
  [frame_clause]  
)
```

PARTITION BY splits rows into independent groups (like GROUP BY, but without collapsing rows); ORDER BY defines the order within each partition for ranking/offset functions; the frame clause controls which rows around the current one are included in the aggregate.

6.2.3 5.2.3 Ranking Functions

-- Schema: employees(full_name, department, salary)

SELECT full_name, department, salary,
ROW_NUMBER() **OVER** (**PARTITION BY** department **ORDER BY** salary **DESC**) **AS** row_num,
RANK() **OVER** (**PARTITION BY** department **ORDER BY** salary **DESC**) **AS** rank_val,
DENSE_RANK() **OVER** (**PARTITION BY** department **ORDER BY** salary **DESC**) **AS** dense_rank_val
FROM employees;

full_name	department	salary	row_num	rank_val	dense_rank_val
Nora Patel	Engineering	185000	1	1	1
Sam Ortiz	Engineering	142000	2	2	2
Liu Wei	Engineering	138000	3	3	3
Maya Johnson	Sales	160000	1	1	1
Tom Becker	Sales	95000	2	2	2
Priya Rao	Sales	88000	3	3	3

ROW_NUMBER() always assigns unique sequential numbers even for ties; RANK() gives tied rows the same rank and skips the next number(s); DENSE_RANK() gives tied rows the same rank with no gap. NTILE(n) divides rows into n roughly-equal buckets; PERCENT_RANK() returns a value from 0 to 1.

6.2.4 5.2.4 Offset Functions

```
-- Schema: orders(user_id, order_date, total_amount)
SELECT user_id, order_date, total_amount,
       LAG(total_amount) OVER (PARTITION BY user_id ORDER BY order_date) AS prev_amount,
       LEAD(total_amount) OVER (PARTITION BY user_id ORDER BY order_date) AS next_amount
FROM orders;
```

LAG(col, n, default) reads a prior row; LEAD(col, n, default) reads a following row; FIRST_VALUE/LAST_VALUE/NTH_VALUE pull a specific row's value from within the current frame.

6.2.5 5.2.5 Aggregate Window Functions — Running Totals

```
-- Schema: orders(order_date, total_amount)
SELECT order_date, total_amount,
       SUM(total_amount) OVER (ORDER BY order_date) AS running_total,
       AVG(total_amount) OVER (ORDER BY order_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS avg_3_rows
FROM orders;
```

Without PARTITION BY, the running total accumulates across the entire result set in order_date order.

6.2.6 5.2.6 ROWS vs RANGE

- ROWS BETWEEN n PRECEDING AND CURRENT ROW counts a fixed number of **physical rows**.
- RANGE BETWEEN ... groups by **logical value** — rows with the same ORDER BY value are treated as a single peer group.
- Default frame (when only ORDER BY is given, no explicit frame): RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.

Warning: LAST_VALUE(col) OVER (ORDER BY col) with the *default* frame almost never returns what people expect — the default frame stops at the current row, so LAST_VALUE just returns the current row's own value. Fix: explicitly widen the frame: LAST_VALUE(col) OVER (ORDER BY col ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING).

6.2.7 5.2.7 Named Windows

```
-- Schema: orders(user_id, order_date, total_amount)
SELECT
```

```

    ROW_NUMBER() OVER w AS rn,
    SUM(total_amount) OVER w AS running_total
FROM orders
WINDOW w AS (PARTITION BY user_id ORDER BY order_date);

```

WINDOW defines a reusable window spec once, referenced by name in multiple functions — avoids repeating the same PARTITION BY/ORDER BY clause.

6.2.8 5.2.8 Window Functions vs GROUP BY

Scenario	Use
Need one summary row per group	GROUP BY
Need every original row plus a computed aggregate	Window function
Need ranking within groups	Window function (RANK, ROW_NUMBER)
Need running totals / moving averages	Window function

6.3 5.3 CASE Expressions

6.3.1 5.3.1 Simple vs Searched CASE

-- Schema: orders(status)

```

SELECT status,
       CASE status
         WHEN 'pending' THEN 'Waiting'
         WHEN 'shipped' THEN 'In transit'
         ELSE 'Other'
       END AS status_label
FROM orders;

```

-- Searched CASE – arbitrary boolean conditions per branch

```

SELECT total_amount,
       CASE
         WHEN total_amount > 200 THEN 'Large'
         WHEN total_amount > 50  THEN 'Medium'
         ELSE 'Small'
       END AS size_bucket
FROM orders;

```

6.3.2 5.3.2 Pivot With CASE (Rows to Columns)

-- Schema: orders(status, total_amount)

```

SELECT
  SUM(CASE WHEN status = 'delivered' THEN total_amount ELSE 0 END) AS delivered_total,
  SUM(CASE WHEN status = 'pending'   THEN total_amount ELSE 0 END) AS pending_total,
  SUM(CASE WHEN status = 'cancelled'  THEN total_amount ELSE 0 END) AS cancelled_total
FROM orders;

```

delivered_total	pending_total	cancelled_total
488.97	199.00	8.99

Note: All CASE branches must return compatible types; a missing ELSE implicitly returns NULL.

6.4 5.4 NULL Handling Functions

```
-- Schema: users(full_name)
SELECT COALESCE(full_name, 'Unknown') FROM users; -- first non-NULL argument

-- Schema: orders(total_amount) - avoid divide-by-zero
SELECT total_amount / NULLIF(0, 0) AS safe_division FROM orders; -- NULLIF(a,b): NULL if

SELECT GREATEST(10, 25, 3) AS max_val, LEAST(10, 25, 3) AS min_val; -- 25, 3

-- Schema: users(country) - IS DISTINCT FROM treats NULL as comparable (PostgreSQL)
SELECT * FROM users WHERE country IS DISTINCT FROM 'US';
```

Note: a IS DISTINCT FROM b behaves like <> but never returns UNKNOWN — two NULLs are considered *not* distinct (i.e. equal for this purpose), unlike plain <>.

6.5 5.5 Date/Time Functions Across Dialects

Operation	PostgreSQL	MySQL	SQLite
Now	NOW(), CURRENT_TIMESTAMP	NOW()	datetime('now')
Current date	CURRENT_DATE	CURDATE()	date('now')
Extract year	EXTRACT(YEAR FROM d)	YEAR(d)	strftime('%Y', d)
Truncate to month	DATE_TRUNC('month', d)	DATE_FORMAT(d, '%Y-%m-01')	strftime('%Y-%m- 01', d)
Add 7 days	d + INTERVAL '7 days'	DATE_ADD(d, INTERVAL 7 DAY)	date(d, '+7 days')
Diff in days	d1 - d2	DATEDIFF(d1, d2)	julianday(d1) - julianday(d2)
Format	TO_CHAR(d, 'YYYY-MM-DD')	DATE_FORMAT(d, '%Y-%m-%d')	strftime('%Y-%m- %d', d)

```
-- Schema: orders(order_date)
SELECT order_date, EXTRACT(MONTH FROM order_date) AS order_month
FROM orders; -- PostgreSQL
```

Warning: MySQL TIMESTAMP only covers 1970-2038 [verify]; use DATETIME for dates outside that range. SQLite has no native timezone type — timestamps are just TEXT and comparisons only work correctly if every value uses the same timezone convention. PostgreSQL's AT TIME ZONE converts between zones on TIMESTAMPTZ; MySQL uses CONVERT_TZ().

6.6 5.6 String Functions

```
-- Schema: users(full_name, email)
SELECT full_name || ' <' || email || ' >' AS formatted FROM users; -- PostgreSQL/SQLite: |
SELECT CONCAT(full_name, ' <', email, ' >') FROM users; -- all three dialects

SELECT LENGTH(email) FROM users; -- character/byte count
SELECT UPPER(full_name), LOWER(full_name) FROM users;
SELECT TRIM(full_name) FROM users;

-- PostgreSQL / SQLite: SUBSTRING(str FROM start FOR len) or SUBSTR(str, start, len)
SELECT SUBSTR(email, 1, 5) AS prefix FROM users;

-- Position: POSITION/STRPOS (PG), LOCATE/INSTR (MySQL), INSTR (SQLite)
SELECT POSITION('@' IN email) FROM users; -- PostgreSQL

SELECT REPLACE(email, '@example.com', '@corp.com') FROM users;

-- Split: SPLIT_PART (PG), SUBSTRING_INDEX (MySQL), no native split in SQLite
SELECT SPLIT_PART(email, '@', 1) AS local_part FROM users; -- PostgreSQL

-- Aggregation across rows
SELECT STRING_AGG(full_name, ', ' ORDER BY full_name) FROM users; -- PostgreSQL, SQLite
SELECT GROUP_CONCAT(full_name ORDER BY full_name SEPARATOR ', ') FROM users; -- MySQL
```

Note: LENGTH() measures bytes in some contexts vs characters in others depending on encoding — use CHAR_LENGTH() when you specifically need character count in a multi-byte-safe way.

6.7 5.7 JSON Functions

6.7.1 5.7.1 PostgreSQL JSONB

```
-- Schema (hypothetical): products(product_id, attributes JSONB)
SELECT attributes -> 'color' AS color_json, -- returns JSON
       attributes ->> 'color' AS color_text -- returns text
FROM products;

SELECT * FROM products WHERE attributes @> '{"color": "black"}'; -- containment
SELECT * FROM products WHERE attributes ? 'color'; -- key exists

SELECT jsonb_build_object('id', product_id, 'name', name) FROM products;
-- returns a JSON value, ->> returns text; @>/<@ test containment; ?/?|/?& test key existence
(single/any/all). Index JSONB columns with GIN for fast containment queries.
```

6.7.2 5.7.2 MySQL JSON

```
SELECT JSON_EXTRACT(attributes, '$.color') FROM products; -- 5.7+ [verify]
SELECT attributes->>'$.color' FROM products; -- shorthand
```

6.7.3 5.7.3 SQLite JSON1 Extension

```
SELECT json_extract(attributes, '$.color') FROM products;
```

Note: SQLite has no native JSON column type — json1 functions operate on ordinary TEXT columns containing JSON strings.

6.8 5.8 Full-Text Search

-- PostgreSQL

```
SELECT * FROM products
WHERE to_tsvector('english', name) @@ to_tsquery('english', 'wireless & mouse');
```

-- MySQL – requires a FULLTEXT index

-- ALTER TABLE products ADD FULLTEXT(name);

```
SELECT * FROM products WHERE MATCH(name) AGAINST('wireless mouse' IN NATURAL LANGUAGE MODE);
```

-- SQLite FTS5 – requires a virtual table

-- CREATE VIRTUAL TABLE products_fts USING fts5(name);

```
SELECT * FROM products_fts WHERE products_fts MATCH 'wireless mouse';
```

PostgreSQL ranks results with ts_rank() and highlights matches with ts_headline(); SQLite's FTS5 has BM25 ranking built in [verify]; MySQL's boolean mode (AGAINST('+cat -dog' IN BOOLEAN MODE)) supports required/excluded terms.

7 PART 6: TRANSACTIONS & CONCURRENCY

Canonical schema: users(user_id, email, full_name, created_at, country, is_active) · products(product_id, name, category, price, stock, created_at) · orders(order_id, user_id, order_date, status, total_amount) · order_items(item_id, order_id, product_id, quantity, unit_price) · employees(employee_id, full_name, manager_id, department, salary, hire_date) · departments(department_id, name, budget)

7.1 6.1 ACID Properties

-- Atomicity example: a "transfer" must debit one account and credit another as one unit
-- Schema: accounts(account_id, balance) – hypothetical

```
BEGIN;
```

```
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
```

```
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
```

```
COMMIT;
```

- **Atomicity:** both UPDATES succeed or neither does — no partial transfer.
- **Consistency:** the database moves from one valid state to another; constraints (CHECK, FOREIGN KEY) are enforced throughout.
- **Isolation:** concurrent transactions don't see each other's uncommitted changes (degree controlled by isolation level — see 6.3).
- **Durability:** once COMMIT returns, the change survives a crash, typically via a write-ahead log (WAL).

7.2 6.2 Transaction Control

```
BEGIN;
```

```
START TRANSACTION;
```

-- PostgreSQL, SQLite

-- MySQL, also valid in PostgreSQL

```

SAVEPOINT sp1;
-- ... some work ...
ROLLBACK TO SAVEPOINT sp1;      -- undo back to the savepoint, transaction stays open
RELEASE SAVEPOINT sp1;          -- discard the savepoint marker

COMMIT;      -- persist everything since BEGIN
ROLLBACK;    -- discard everything since BEGIN

```

Warning: Most client libraries default to **autocommit** — each statement is its own implicit transaction unless you explicitly BEGIN. Some ORMs and GUI tools silently keep autocommit on even inside what looks like a transactional block; verify your driver’s default before assuming statements are grouped.

7.3 6.3 Isolation Levels

7.3.1 6.3.1 Read Phenomena

- **Dirty read:** transaction A reads a row transaction B has modified but not yet committed.
- **Non-repeatable read:** A re-reads the same row within one transaction and gets a different value because B committed a change in between.
- **Phantom read:** A re-runs the same range query and sees new rows that B inserted in between.
- **Lost update:** A and B both read the same row, both write back based on the stale value, and one write silently overwrites the other.

7.3.2 6.3.2 Isolation Level Comparison

Level	Dirty Read	Non-Repeatable Read	Phantom Read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

7.3.3 6.3.3 Defaults Per Dialect

- PostgreSQL: READ COMMITTED.
- MySQL InnoDB: REPEATABLE READ.
- SQLite: effectively SERIALIZABLE via whole-database locking by default.

```

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN;
-- ...
COMMIT;

```

Note: Higher isolation levels prevent more anomalies but increase the chance of transactions being forced to abort and retry (especially under SERIALIZABLE), trading concurrency for correctness guarantees.

7.4 6.4 Locking

```

-- Schema: orders(order_id, status)
BEGIN;

```

```
SELECT * FROM orders WHERE order_id = 1 FOR UPDATE;  -- exclusive lock, held until COMMIT
-- ... process the order ...
COMMIT;
```

Variant	Behavior
FOR UPDATE	Exclusive lock — blocks other writers and other FOR UPDATE readers
FOR SHARE	Shared lock — allows other shared locks, blocks writers
NOWAIT	Fail immediately instead of waiting if the row is locked
SKIP LOCKED	Skip already-locked rows — used to build a job queue where multiple workers each grab a different row

```
-- Queue pattern: each worker grabs one unclaimed job without blocking on others
-- Schema: jobs(job_id, status) – hypothetical
BEGIN;
SELECT * FROM jobs WHERE status = 'pending'
ORDER BY job_id LIMIT 1 FOR UPDATE SKIP LOCKED;
```

Best Practice: Row-level locks (the default granularity in PostgreSQL/MySQL InnoDB) allow much higher concurrency than table-level locks — but every lock still costs bookkeeping overhead, so don't lock rows you don't need to modify.

7.5 6.5 Deadlocks

A deadlock occurs when transaction A holds a lock B needs, and B holds a lock A needs — neither can proceed.

```
-- Transaction A
BEGIN;
UPDATE accounts SET balance=balance-10
WHERE account_id = 1;
UPDATE accounts SET balance=balance+10
WHERE account_id = 2; -- blocks on B

-- Transaction B
BEGIN;
UPDATE accounts SET balance=balance-10
WHERE account_id = 2;
UPDATE accounts SET balance=balance+10
WHERE account_id = 1; -- blocks on A → deadlock
```

The database detects the cycle and kills one transaction (rolling it back with an error) so the other can proceed.

Recommended: Always acquire locks in the same order across every transaction that touches the same rows (e.g. always update the lower account_id first) — this single discipline eliminates most application-level deadlocks.

- PostgreSQL: logs deadlocks; tune deadlock_timeout for detection latency.
- MySQL: SHOW ENGINE INNODB STATUS shows the most recent deadlock's details.

7.6 6.6 MVCC (Multi-Version Concurrency Control)

Each transaction sees a consistent snapshot of the database as of when it started — writers don't block readers, and readers don't block writers.

- **PostgreSQL:** every row version carries hidden xmin/xmax columns marking which transaction created/deleted it; old versions become “dead tuples” that VACUUM must reclaim, or the table bloats.

- **MySQL InnoDB:** keeps previous row versions in undo logs; a background purge thread cleans them up once no transaction still needs them.

Warning: A long-running transaction in PostgreSQL prevents VACUUM from reclaiming dead tuples created after it started, which can cause significant table bloat under high write volume.

8 PART 7: PERFORMANCE & OPTIMIZATION

8.1 7.1 Query Execution Plans

8.1.1 7.1.1 EXPLAIN vs EXPLAIN ANALYZE

```
-- Schema: orders(user_id, status)
EXPLAIN SELECT * FROM orders WHERE user_id = 1;      -- estimated plan, doesn't run
EXPLAIN ANALYZE SELECT * FROM orders WHERE user_id = 1;  -- actually runs it, shows re
```

Warning: EXPLAIN ANALYZE executes the query for real, including any INSERT/UPDATE/DELETE side effects — wrap mutating statements in a transaction you roll back if you’re only profiling.

8.1.2 7.1.2 Reading a Plan

A PostgreSQL plan node looks like: Index Scan using idx_orders_user_id on orders (cost=0.15..8.17 rows=1 width=42) (actual time=0.02..0.03 rows=1 loops=1).

- cost=0.15..8.17 — estimated startup cost .. total cost, in arbitrary planner units.
- rows=1 — estimated row count.
- actual time=... rows=1 loops=1 — only present with ANALYZE: real elapsed time and row count.

8.1.3 7.1.3 Scan and Join Types

Scan type	When used
Seq Scan	Full table scan — expected for small tables or when no useful index exists
Index Scan	Uses an index to seek directly to matching rows
Index Only Scan	The index alone contains every column the query needs — no heap access
Bitmap Heap Scan	Batches many index matches together — good when returning a moderate-to-large fraction of rows

Join type	Chosen when
Nested Loop	One side is small — for each outer row, scan/seek the inner
Hash Join	Build a hash table on the smaller input, probe it with the larger

Join type	Chosen when
Merge Join	Both inputs are already sorted on the join key

Best Practice: A Seq Scan on a large table with a highly selective WHERE filter is the single most common sign of a missing index. A large gap between estimated and actual row counts signals stale statistics — run ANALYZE (PostgreSQL) or update MySQL's statistics.

8.2 7.2 Indexing Strategies

```
-- Schema: orders(user_id, order_date)
-- Composite index: order of columns matters (leftmost-prefix rule)
CREATE INDEX idx_orders_user_date ON orders (user_id, order_date);
-- Supports: WHERE user_id = ?           (uses the index)
-- Supports: WHERE user_id = ? AND order_date = ? (uses the index fully)
-- Does NOT support: WHERE order_date = ? alone (can't skip the leading column)
-- Covering index (PostgreSQL): includes extra columns without adding them to the key
CREATE INDEX idx_orders_covering ON orders (user_id) INCLUDE (total_amount);

-- Partial index: smaller, faster, only helps queries with a matching WHERE
CREATE INDEX idx_active_users ON users (email) WHERE is_active = TRUE;

-- Functional index: must match the exact expression used in the query
CREATE INDEX idx_users_email_lower ON users (LOWER(email));
-- Only helps: WHERE LOWER(email) = 'x' - NOT: WHERE email = 'x'

-- Find unused indexes (PostgreSQL)
SELECT indexrelname, idx_scan FROM pg_stat_user_indexes WHERE idx_scan = 0;
```

Maintenance	PostgreSQL	MySQL
Rebuild index	REINDEX	—
Refresh statistics	VACUUM ANALYZE	ANALYZE TABLE
Reclaim space	VACUUM FULL	OPTIMIZE TABLE

Anti-Pattern: Indexing every column “just in case.” Each index slows down every INSERT/UPDATE/DELETE that touches it and consumes disk space — low-cardinality columns (booleans, small enums) rarely benefit at all.

8.3 7.3 Query Optimization Patterns

```
-- AVOID: Avoid SELECT * - fetches unneeded columns, blocks index-only scans
SELECT * FROM orders WHERE user_id = 1;
-- BETTER: Select only what you need
SELECT order_id, total_amount FROM orders WHERE user_id = 1;

-- AVOID: A function on an indexed column prevents index use
SELECT * FROM users WHERE LOWER(email) = 'alice@example.com';
-- BETTER: Use a functional index (7.2) or store a normalized column instead

-- AVOID: OR across different columns often can't use either index
SELECT * FROM orders WHERE user_id = 1 OR status = 'pending';
```

-- BETTER: Rewrite as a UNION of two indexed queries

```
SELECT * FROM orders WHERE user_id = 1
UNION
SELECT * FROM orders WHERE status = 'pending';
```

Best Practice: LIMIT combined with an ORDER BY on an indexed column lets the planner stop scanning as soon as it has enough rows, instead of sorting the entire result set.

8.4 7.4 Common Anti-Patterns

-- AVOID: N+1: one query per user in application code (pseudocode)

-- for user in users: db.query("SELECT * FROM orders WHERE user_id = ?", user.id)

-- BETTER: Fix: one query with IN or a JOIN

```
SELECT * FROM orders WHERE user_id IN (1, 2, 3, 4, 5);
```

-- AVOID: Leading wildcard can't use a B-tree index

```
SELECT * FROM products WHERE name LIKE '%mouse%';
```

-- BETTER: Use full-text search or a trigram index (pg_trgm) instead

-- AVOID: Deep OFFSET pagination re-scans and discards every prior row

```
SELECT * FROM orders ORDER BY order_id LIMIT 10 OFFSET 10000;
```

-- BETTER: Keyset pagination – jump straight to where you left off

```
SELECT * FROM orders WHERE order_id > 10000 ORDER BY order_id LIMIT 10;
```

Anti-pattern	Fix
Implicit type conversion (WHERE user_id = '42' when user_id is INT)	Match types explicitly
Cartesian product from a missing/wrong JOIN condition	Always specify the join key; watch for unexpectedly huge row counts
SELECT DISTINCT used to “hide” duplicate rows	Fix the underlying join instead of masking the symptom
Storing dates as strings	Use native DATE/TIMESTAMP types so range queries and sorting work correctly
Storing comma-separated values in a column	Normalize into a junction table (violates 1NF otherwise)
No primary key on a table	Always define one — needed for replication, deduplication, and safe row identification

9 PART 8: DIALECT DIFFERENCES APPENDIX

Canonical schema: users(user_id, email, full_name, created_at, country, is_active) · products(product_id, name, category, price, stock, created_at) · orders(order_id, user_id, order_date, status, total_amount) · order_items(item_id, order_id, product_id, quantity, unit_price) · employees(employee_id, full_name, manager_id, department, salary, hire_date) · departments(department_id, name, budget)

9.1 8.1 Data Types

Concept	PostgreSQL	MySQL	SQLite
Small int	SMALLINT	SMALLINT	INTEGER
Int	INTEGER	INT	INTEGER
Big int	BIGINT	BIGINT	INTEGER
Auto-increment	SERIAL / IDENTITY	AUTO_INCREMENT	AUTOINCREMENT
Exact decimal	NUMERIC(p, s)	DECIMAL(p, s)	NUMERIC
Float	REAL, DOUBLE PRECISION	FLOAT, DOUBLE	REAL
Boolean	BOOLEAN	TINYINT(1)	INTEGER (0/1)
Char	CHAR(n)	CHAR(n)	TEXT
Varchar	VARCHAR(n)	VARCHAR(n)	TEXT
Text	TEXT	TEXT/LONGTEXT	TEXT
Date	DATE	DATE	TEXT
Timestamp	TIMESTAMP/TIMESTAMPTZ	DATETIME/TIMESTAMP	TEXT
JSON	JSON, JSONB	JSON [verify 5.7+]	TEXT (json1 ext.)
UUID	UUID (native)	CHAR(36)	TEXT
Array	✓ native	✗	✗
Enum	✓ native	✓ native	✗ (emulate with CHECK)

9.2 8.2 DDL Syntax

Operation	PostgreSQL	MySQL	SQLite
Add column	ALTER TABLE t ADD COLUMN c type	ALTER TABLE t ADD c type	ALTER TABLE t ADD COLUMN c type
Drop column	ALTER TABLE t DROP COLUMN c	ALTER TABLE t DROP c	ALTER TABLE t DROP COLUMN c (3.35+ [verify])
Rename column	RENAME COLUMN a TO b	RENAME COLUMN a TO b (8.0+ [verify])	RENAME COLUMN a TO b (3.25+ [verify])
Modify type	ALTER COLUMN c TYPE t	MODIFY c t	not supported — recreate table
Rename table	RENAME TO t2	RENAME TABLE t TO t2	RENAME TO t2
Truncate	TRUNCATE TABLE t	TRUNCATE TABLE t	DELETE FROM t (no native truncate)
Partial index	✓ WHERE clause	✗	✓ WHERE clause
Materialized view	✓	✗	✗

9.3 8.3 DML Syntax

Operation	PostgreSQL	MySQL	SQLite
Upsert	ON CONFLICT (...) DO UPDATE	ON DUPLICATE KEY UPDATE	ON CONFLICT (...) DO UPDATE
Returning generated ID	RETURNING *	LAST_INSERT_ID()	RETURNING * (3.35+ [verify])
Multi-table UPDATE	UPDATE ... FROM	UPDATE ... JOIN	not supported
Multi-table DELETE	DELETE ... USING	DELETE t1 FROM t1 JOIN t2	not supported

9.4 8.4 Function Comparisons

Category	PostgreSQL	MySQL	SQLite
Concat	or CONCAT()	CONCAT()	
Substring	SUBSTRING(s FROM a FOR b)	SUBSTRING(s,a,b)	SUBSTR(s,a,b)
Position	POSITION, STRPOS	LOCATE, INSTR	INSTR
Split	SPLIT_PART	SUBSTRING_INDEX	none native
String agg	STRING_AGG	GROUP_CONCAT	GROUP_CONCAT
Regex	~, regexp_replace()	REGEXP_LIKE (8.0+ [verify])	none native
Extract year	EXTRACT(YEAR FROM d)	YEAR(d)	strftime('%Y', d)
Date truncate	DATE_TRUNC	DATE_FORMAT	strftime
Add interval	d + INTERVAL '1 day'	DATE_ADD(d, INTERVAL 1 DAY)	date(d, '+1 day')
Date diff	d1 - d2	DATEDIFF(d1,d2)	julianday(d1) - julianday(d2)

9.5 8.5 Window Functions & CTEs

Feature	PostgreSQL	MySQL	SQLite
Window functions	✓ (8.4+ [verify])	✓ (8.0+ [verify])	✓ (3.25+ [verify])
Named windows (WINDOW)	✓	✓	✓
FILTER clause	✓	✗	✓
Basic WITH	✓	✓ (8.0+ [verify])	✓ (3.8.3+ [verify])
WITH RECURSIVE	✓	✓	✓
MATERIALIZED hint	✓ (12+ [verify])	✗	✗

9.6 8.6 Error Handling

Feature	PostgreSQL	MySQL	SQLite
Try/catch	PL/pgSQL BEGIN ... EXCEPTION	DECLARE ... HANDLER	not supported
Error identifiers	SQLSTATE	SQLSTATE + engine error number	sqlite_errmsg()

10 PART 9: INTERVIEW PREPARATION

Canonical schema: users(user_id, email, full_name, created_at, country, is_active) · products(product_id, name, category, price, stock, created_at) · orders(order_id, user_id, order_date, status, total_amount) · order_items(item_id, order_id, product_id, quantity, unit_price) · employees(employee_id, full_name, manager_id, department, salary, hire_date) · departments(department_id, name, budget)

10.1 9.1 Top 50 SQL Interview Questions

10.1.1 Beginner

1. What is SQL, and how does it differ from NoSQL? SQL is a declarative language for relational (table-based) databases with a fixed schema and strong consistency guarantees (ACID). NoSQL databases (document, key-value, wide-column, graph) trade schema rigidity and sometimes strict consistency for horizontal scalability and flexible data shapes.

2. What is a primary key? A column (or set of columns) that uniquely identifies each row and cannot be NULL. Example: `user_id SERIAL PRIMARY KEY` in `users`.

3. What is a foreign key? A column that references another table's primary key, enforcing that the referenced value must exist. Example: `orders.user_id REFERENCES users(user_id)`. **X** Common wrong answer: "it's just any column that links two tables" — it specifically enforces referential integrity, it doesn't just associate them informally.

4. Difference between WHERE and HAVING? WHERE filters individual rows before grouping and can't use aggregates; HAVING filters groups after GROUP BY and can use aggregates like SUM().

5. What is a JOIN? List the types. A JOIN combines rows from two or more tables based on a related column. Types: INNER, LEFT, RIGHT, FULL OUTER, CROSS, plus the special cases SELF and NATURAL (see Part 4.5).

6. Difference between INNER JOIN and LEFT JOIN? INNER JOIN returns only rows with a match in both tables; LEFT JOIN returns all rows from the left table plus matches from the right, filling NULL where there's no match.

7. What is a self-join? Give an example. A table joined to itself via two aliases — the standard example is `employees e JOIN employees m ON e.manager_id = m.employee_id` to resolve each employee's manager name.

8. Difference between UNION and UNION ALL? UNION deduplicates the combined result (slower, does an implicit sort/hash); UNION ALL keeps every row including duplicates (faster). Default to UNION ALL unless you need dedup.

9. What is a subquery? What types exist? A query nested inside another query. Types: scalar (one value), row (one row, multiple columns), table/derived (used in FROM), and correlated (references the outer query).

10. What is a correlated subquery? A subquery that references a column from the outer query, so it conceptually re-evaluates once per outer row — e.g. `WHERE EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.user_id)`.

11. Difference between DELETE, TRUNCATE, and DROP? DELETE removes rows (optionally filtered, triggers fire, rollback-able); TRUNCATE removes all rows fast (no WHERE, no triggers); DROP removes the entire table structure. See the comparison table in 2.4.

12. What is an index? What types exist? A separate data structure that speeds up lookups on a column at the cost of extra writes and storage. B-tree is the default; PostgreSQL also offers GIN, GiST, BRIN, and hash indexes for specialized cases.

13. What does GROUP BY do? Collapses rows sharing the same value(s) in the grouped column(s) into a single row per group, typically paired with aggregate functions.

****14. Difference between COUNT(*) and COUNT(column)?**** COUNT(*) counts every row; COUNT(column) counts only rows where that column is non-NULL.

15. What is NULL? How does it differ from an empty string? NULL means "unknown" or "absent" — it's not a value at all, and comparisons against it return UNKNOWN. An empty string '' is a real, zero-length value that equals other empty strings.

16. What is a view? What are its use cases? A named, stored query that acts like a virtual table — no data is stored (see materialized views for that). Used to simplify repeated complex queries or to restrict which columns/rows a user can see.

17. Difference between CHAR and VARCHAR? CHAR(n) is fixed-length and pads with trailing spaces; VARCHAR(n) is variable-length and stores exactly what's given, up to the limit.

18. What is a composite key? A primary (or unique) key made of more than one column — common in junction tables, e.g. PRIMARY KEY (order_id, product_id).

19. Explain DISTINCT. SELECT DISTINCT removes duplicate rows from the result set based on all selected columns together, not per-column.

20. How do you find duplicate rows?

```
-- Schema: users(email)
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;
```

10.1.2 Intermediate

21. Write a query to find the second-highest salary.

```
-- Schema: employees(salary)
SELECT MAX(salary) AS second_highest FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

22. How do you find the Nth highest salary? Use DENSE_RANK() (see 9.2.1 for all three approaches):

```
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees
) t WHERE rnk = 3;
```

23. Find employees earning more than their manager.

```
-- Schema: employees(employee_id, manager_id, salary)
SELECT e.full_name FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;
```

24. Delete duplicate rows, keeping one. See 9.2.3 for the full ROW_NUMBER() pattern.

25. Difference between RANK, DENSE_RANK, and ROW_NUMBER? ROW_NUMBER() always assigns unique numbers even for ties; RANK() gives ties the same rank and skips subsequent numbers; DENSE_RANK() gives ties the same rank with no gap. See 5.2.3 for a side-by-side example.

26. How do you find gaps in a numeric sequence? See 9.2.6.

27. Explain CTEs and give an example. A WITH clause names an intermediate result for readability and reuse within a single statement — see Part 5.1.

28. What is a recursive CTE? A CTE that references itself via WITH RECURSIVE, used for hierarchical or graph-like data such as org charts (5.1.5) or number generation (5.1.6).

29. Difference between a subquery and a CTE? Functionally similar in many cases, but a CTE is named and can be referenced multiple times in the outer query without repeating its SQL, and reads top-to-bottom rather than nesting inward.

30. What are window functions? Give an example. Functions that compute a value across a set of related rows (defined by OVER (...)) without collapsing them into one row

per group, unlike GROUP BY. Example: SUM(total_amount) OVER (PARTITION BY user_id) — see Part 5.2.

31. Difference between ROWS and RANGE in window frames? ROWS counts a fixed number of physical rows; RANGE groups rows with the same ORDER BY value as peers. See 5.2.6 for the LAST_VALUE gotcha this causes.

32. What is an upsert? An insert that falls back to an update when a conflicting unique/primary key already exists: INSERT ... ON CONFLICT DO UPDATE (PostgreSQL/SQLite) or INSERT ... ON DUPLICATE KEY UPDATE (MySQL).

33. How do you pivot rows into columns in SQL? With conditional aggregation using CASE inside SUM/COUNT — see 5.3.2.

34. Difference between EXISTS and IN? Both test membership, but EXISTS short-circuits at the first match (often faster for large subqueries) and is NULL-safe, while NOT IN silently returns zero rows if the subquery contains any NULL (see 4.4).

35. What is a materialized view? A view whose results are physically stored and must be explicitly refreshed (REFRESH MATERIALIZED VIEW), trading freshness for read speed — native to PostgreSQL, not MySQL.

10.1.3 Advanced

36. Explain the ACID properties. See Part 6.1 for all four with a transfer example.

37. What are isolation levels and how do they differ? See 6.3 — four standard levels trading anomaly prevention against concurrency.

38. What is a dirty read? A phantom read? A dirty read sees another transaction's uncommitted changes; a phantom read sees new rows appear on a repeated range query because another transaction inserted rows in between (6.3.1).

39. How do you detect and resolve deadlocks? The database automatically detects the wait cycle and kills one transaction; prevention means always acquiring locks on shared resources in the same order across transactions (6.5).

40. What is MVCC? Multi-Version Concurrency Control — each transaction reads a consistent snapshot, so readers never block writers and vice versa. PostgreSQL implements it via xmin/xmax and VACUUM; MySQL InnoDB via undo logs (6.6).

41. How do you read an EXPLAIN plan? Look at scan/join node types, the cost and row estimates, and (with ANALYZE) the actual timings and row counts — large estimate-vs-actual gaps point to stale statistics (7.1).

42. Explain Seq Scan vs Index Scan vs Bitmap Scan. A Seq Scan reads every row; an Index Scan seeks directly via an index; a Bitmap Heap Scan batches many index matches before visiting the table — efficient for a moderate fraction of matching rows (7.1.3).

43. What is a covering index? An index that contains every column a query needs, letting the engine skip the underlying table entirely (an “index-only scan” in PostgreSQL). See 7.2.

44. What is a partial index? An index built over only the rows matching a WHERE condition, smaller and faster than a full index but only usable by queries with a matching filter (7.2).

45. Why might my index not be used? Common causes: a function wraps the indexed column (WHERE LOWER(email)=... without a matching functional index), a leading wildcard LIKE '%x', an implicit type mismatch, low table selectivity, or stale planner statistics.

46. How do you optimize a slow query? Run EXPLAIN ANALYZE, check for missing indexes on filter/join columns, avoid SELECT *, push filters as early as possible, and check whether the query can be rewritten to avoid functions on indexed columns (Part 7.3).

47. What is the N+1 problem? How do you fix it? Issuing one query per row in a loop instead of a single set-based query — fix by batching with IN (...) or a JOIN (7.4).

48. Explain Hash Join, Merge Join, and Nested Loop. Nested loop scans the inner table once per outer row (good for small outer sets); hash join builds a hash table on the smaller side and probes with the larger (good for large unsorted inputs); merge join sorts both sides and merges them (good when inputs are already sorted) — 7.1.3.

49. How do you handle slow OFFSET pagination? Switch to keyset pagination: WHERE id > last_seen_id ORDER BY id LIMIT n, which avoids re-scanning and discarding all prior rows (7.4).

50. How do you scale a database for reads and writes? Reads: add read replicas, caching layers, and materialized views. Writes: vertical scaling first, then sharding/partitioning by a key, or moving write-heavy workloads to a queue-backed async model. Indexing and query optimization (Part 7) should come before any of these.

10.2 9.2 Classic Problems With Runnable Solutions

10.2.1 9.2.1 Nth Highest Salary — Three Approaches

```
-- Schema: employees(salary)
-- Approach 1: LIMIT/OFFSET (simple, but slow on large tables without an index, and
-- mishandles ties — two employees tied for 2nd will only count once)
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 2; -- 3rd highest

-- Approach 2: DENSE_RANK (correctly handles ties as one rank)
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees
) t WHERE rnk = 3;

-- Approach 3: Correlated subquery (portable to engines without window functions)
SELECT DISTINCT salary FROM employees e1
WHERE 2 = (SELECT COUNT(DISTINCT salary) FROM employees e2 WHERE e2.salary > e1.salary);
```

salary
138000

Best Practice: Prefer the DENSE_RANK() approach — it's the clearest about tie-handling and performs a single pass over the data.

10.2.2 9.2.2 Find Duplicate Records

```
-- Schema: users(email)
SELECT email, COUNT(*) AS occurrences FROM users GROUP BY email HAVING COUNT(*) > 1;
```

10.2.3 9.2.3 Delete Duplicate Rows, Keeping One

```
-- Schema: users(user_id, email) — PostgreSQL, using ROW_NUMBER over a CTE
WITH ranked AS (
```

```

SELECT user_id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY user_id) AS rn
FROM users
)
DELETE FROM users WHERE user_id IN (SELECT user_id FROM ranked WHERE rn > 1);

```

Keeps the lowest user_id per email and deletes the rest. PostgreSQL can alternatively use the hidden ctid column instead of a surrogate key.

10.2.4 9.2.4 Employees Earning More Than Their Manager

```

-- Schema: employees(employee_id, full_name, manager_id, salary)
SELECT e.full_name, e.salary, m.full_name AS manager, m.salary AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;

```

10.2.5 9.2.5 Running Totals and Moving Averages

```

-- Schema: orders(order_date, total_amount)
SELECT order_date, total_amount,
       SUM(total_amount) OVER (ORDER BY order_date) AS running_total,
       AVG(total_amount) OVER (ORDER BY order_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS moving_avg
FROM orders ORDER BY order_date;

```

10.2.6 9.2.6 Find Gaps in a Sequence

```

-- Schema: seq(id) – any table with an integer id column
SELECT id + 1 AS gap_start, next_id - 1 AS gap_end
FROM (
  SELECT id, LEAD(id) OVER (ORDER BY id) AS next_id FROM seq
) t
WHERE next_id - id > 1;

```

10.2.7 9.2.7 Pivot Rows to Columns With CASE

```

-- Schema: orders(user_id, status, total_amount)
SELECT user_id,
       SUM(CASE WHEN status = 'delivered' THEN total_amount ELSE 0 END) AS delivered,
       SUM(CASE WHEN status = 'pending' THEN total_amount ELSE 0 END) AS pending
FROM orders GROUP BY user_id;

```

10.2.8 9.2.8 Hierarchical Query With Recursive CTE

Full org chart — identical to 5.1.5:

```

-- Schema: employees(employee_id, full_name, manager_id)
WITH RECURSIVE org_chart AS (
  SELECT employee_id, full_name, manager_id, 1 AS level
  FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.employee_id, e.full_name, e.manager_id, oc.level + 1
  FROM employees e JOIN org_chart oc ON e.manager_id = oc.employee_id
)
SELECT * FROM org_chart ORDER BY level;

```

10.3 9.3 Query Pattern Cookbook

10.3.1 9.3.1 Top N Per Group

```
-- Schema: products(category, name, price)
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY category ORDER BY price DESC) AS rn
  FROM products
) t WHERE rn <= 3;
```

10.3.2 9.3.2 Year-Over-Year Comparison

```
-- Schema: yearly_revenue(year, revenue) – hypothetical aggregate table
SELECT year, revenue,
  LAG(revenue) OVER (ORDER BY year) AS prev_year,
  revenue - LAG(revenue) OVER (ORDER BY year) AS yoy_change
FROM yearly_revenue;
```

10.3.3 9.3.3 Month-Over-Month Growth

```
-- Schema: orders(order_date, total_amount)
WITH monthly AS (
  SELECT DATE_TRUNC('month', order_date) AS month, SUM(total_amount) AS revenue
  FROM orders GROUP BY 1
)
SELECT month, revenue,
  (revenue - LAG(revenue) OVER (ORDER BY month)) / NULLIF(LAG(revenue) OVER (ORDER BY month), 0) AS growth
FROM monthly;
```

10.3.4 9.3.4 Cohort Analysis (Signup Month)

```
-- Schema: users(user_id, created_at), orders(user_id, order_date)
SELECT DATE_TRUNC('month', u.created_at) AS cohort_month,
  DATE_TRUNC('month', o.order_date) AS activity_month,
  COUNT(DISTINCT u.user_id) AS active_users
FROM users u JOIN orders o ON u.user_id = o.user_id
GROUP BY 1, 2 ORDER BY 1, 2;
```

10.3.5 9.3.5 Retention Analysis (% Active N Days Later)

```
-- Schema: users(user_id, created_at), orders(user_id, order_date)
SELECT COUNT(DISTINCT CASE WHEN o.order_date >= u.created_at + INTERVAL '30 days'
  THEN u.user_id END)::FLOAT / COUNT(DISTINCT u.user_id) AS retention
FROM users u LEFT JOIN orders o ON u.user_id = o.user_id;
```

10.3.6 9.3.6 Funnel Analysis

```
-- Schema (hypothetical): events(user_id, event_name)
SELECT
  COUNT(DISTINCT CASE WHEN event_name = 'view' THEN user_id END) AS viewed,
  COUNT(DISTINCT CASE WHEN event_name = 'add_cart' THEN user_id END) AS added_to_cart,
```

```
COUNT(DISTINCT CASE WHEN event_name = 'purchase' THEN user_id END) AS purchased
FROM events;
```

10.3.7 9.3.7 Sessionization (Gap-Based)

```
-- Schema (hypothetical): events(user_id, event_time)
SELECT user_id, event_time,
       SUM(CASE WHEN event_time - LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time
                                                         > INTERVAL '30 minutes' THEN 1 ELSE 0 END)
           OVER (PARTITION BY user_id ORDER BY event_time) AS session_id
FROM events;
```

A new session starts whenever the gap since the previous event exceeds the threshold (here 30 minutes).

10.3.8 9.3.8 Median (50th Percentile)

```
-- Schema: orders(total_amount) - PostgreSQL
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY total_amount) AS median FROM orders;
```

10.3.9 9.3.9 Percentile Calculation (95th)

```
SELECT PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY total_amount) AS p95 FROM orders;
```

10.3.10 9.3.10 Cumulative Distribution

```
-- Schema: orders(total_amount)
SELECT total_amount, CUME_DIST() OVER (ORDER BY total_amount) AS cume_dist FROM orders;
```

10.3.11 9.3.11 Running Distinct Count

```
-- Schema: orders(user_id, order_date) - PostgreSQL only for windowed DISTINCT aggregates
SELECT order_date, COUNT(DISTINCT user_id) OVER (ORDER BY order_date) AS running_distinct
FROM orders;
```

10.3.12 9.3.12 Rank Within Partition

```
-- Schema: employees(department, salary)
SELECT full_name, department, salary,
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dept_rank
FROM employees;
```

10.3.13 9.3.13 First / Last Value in Group

```
-- Schema: orders(user_id, order_date, total_amount)
SELECT user_id,
       FIRST_VALUE(total_amount) OVER (PARTITION BY user_id ORDER BY order_date) AS first_order
       LAST_VALUE(total_amount) OVER (PARTITION BY user_id ORDER BY order_date
                                       ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_order
FROM orders;
```

10.3.14 9.3.14 Deduplication (Keep Latest)

```
-- Schema: users(user_id, email, created_at)
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY email ORDER BY created_at DESC) AS rn
    FROM users
) t WHERE rn = 1;
```

10.3.15 9.3.15 Slowly Changing Dimension (Type 2 Lookup)

```
-- Schema (hypothetical): product_history(product_id, price, valid_from, valid_to)
SELECT price FROM product_history
WHERE product_id = 1 AND '2026-02-01' BETWEEN valid_from AND COALESCE(valid_to, '9999-12-31');
```

10.3.16 9.3.16 Date Spine Generation

```
-- PostgreSQL: generate_series
SELECT generate_series('2026-01-01'::date, '2026-01-31'::date, '1 day') AS day;
```

10.3.17 9.3.17 Moving Average (7-Day)

```
-- Schema: orders(order_date, total_amount)
SELECT order_date,
    AVG(total_amount) OVER (ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS avg_7d
FROM orders;
```

10.3.18 9.3.18 Recursive Hierarchy Depth

```
-- Schema: employees(employee_id, manager_id) – see 5.1.5 for the full pattern
WITH RECURSIVE depth AS (
    SELECT employee_id, 0 AS lvl FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.employee_id, d.lvl + 1 FROM employees e JOIN depth d ON e.manager_id = d.employee_id
)
SELECT MAX(lvl) AS org_depth FROM depth;
```

10.3.19 9.3.19 Unpivot (Columns to Rows)

```
-- Schema (hypothetical): quarterly(product_id, q1, q2, q3, q4)
SELECT product_id, 'q1' AS quarter, q1 AS revenue FROM quarterly
UNION ALL SELECT product_id, 'q2', q2 FROM quarterly
UNION ALL SELECT product_id, 'q3', q3 FROM quarterly
UNION ALL SELECT product_id, 'q4', q4 FROM quarterly;
```

10.3.20 9.3.20 Gap Detection (Missing Dates)

```
-- Schema: orders(order_date) – find dates with zero orders in a date spine
SELECT d.day FROM generate_series('2026-01-01'::date, '2026-01-31'::date, '1 day') AS d(d.day)
LEFT JOIN orders o ON o.order_date = d.day
WHERE o.order_date IS NULL;
```

10.3.21 9.3.21 Island Detection (Consecutive Runs)

```
-- Schema (hypothetical): logins(user_id, login_date)
-- Group consecutive dates into "islands" using the date-minus-row-number trick
SELECT user_id, MIN(login_date) AS island_start, MAX(login_date) AS island_end
FROM (
    SELECT user_id, login_date,
           login_date - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date))::int AS grp
    FROM logins
) t GROUP BY user_id, grp;
```

10.3.22 9.3.22 Consecutive Login Streaks

Reuses 9.3.21's grp column — the longest streak per user is MAX(island_end - island_start + 1).

10.3.23 9.3.23 Cohort Retention Matrix

```
-- Schema: users(user_id, created_at), orders(user_id, order_date)
SELECT DATE_TRUNC('month', u.created_at) AS cohort,
       DATE_PART('month', AGE(o.order_date, u.created_at)) AS months_since_signup,
       COUNT(DISTINCT u.user_id) AS active
FROM users u JOIN orders o ON u.user_id = o.user_id
GROUP BY 1, 2 ORDER BY 1, 2;
```

10.3.24 9.3.24 RFM Analysis (Recency, Frequency, Monetary)

```
-- Schema: orders(user_id, order_date, total_amount)
SELECT user_id,
       CURRENT_DATE - MAX(order_date) AS recency_days,
       COUNT(*) AS frequency,
       SUM(total_amount) AS monetary
FROM orders GROUP BY user_id;
```

10.3.25 9.3.25 Basket Analysis (Products Bought Together)

```
-- Schema: order_items(order_id, product_id)
SELECT a.product_id AS product_a, b.product_id AS product_b, COUNT(*) AS times_together
FROM order_items a
JOIN order_items b ON a.order_id = b.order_id AND a.product_id < b.product_id
GROUP BY 1, 2 ORDER BY times_together DESC;
```

10.3.26 9.3.26 Deduplicate and Merge Records

```
-- Schema: users(user_id, email, full_name) – keep the row with the most complete data
SELECT email, COALESCE(MAX(full_name), 'Unknown') AS full_name
FROM users GROUP BY email;
```

10.3.27 9.3.27 Latest Row Per Group

```
-- Schema: orders(user_id, order_date) – DISTINCT ON is PostgreSQL-specific
SELECT DISTINCT ON (user_id) user_id, order_date, total_amount
FROM orders ORDER BY user_id, order_date DESC;
```

10.3.28 9.3.28 Top-N With Ties Included

```
-- Schema: products(category, price) – RANK includes ties; ROW_NUMBER would cut one off
SELECT * FROM (
  SELECT *, RANK() OVER (PARTITION BY category ORDER BY price DESC) AS rn FROM products
) t WHERE rn <= 3;
```

10.3.29 9.3.29 Percent Change Between Rows

```
-- Schema: orders(order_date, total_amount)
SELECT order_date, total_amount,
  (total_amount - LAG(total_amount) OVER (ORDER BY order_date))
  / NULLIF(LAG(total_amount) OVER (ORDER BY order_date), 0) * 100 AS pct_change
FROM orders;
```

10.3.30 9.3.30 Period-Over-Period Comparison

```
-- Schema: orders(order_date, total_amount) – this week vs last week
SELECT
  SUM(CASE WHEN order_date >= CURRENT_DATE - 7 THEN total_amount ELSE 0 END) AS this_week,
  SUM(CASE WHEN order_date >= CURRENT_DATE - 14 AND order_date < CURRENT_DATE - 7
    THEN total_amount ELSE 0 END) AS last_week
FROM orders;
```

11 PART 10: CHEAT SHEET

11.1 10.1 DDL Quick Reference

```
CREATE TABLE t (id SERIAL PRIMARY KEY, name VARCHAR(100) NOT NULL, created_at TIMESTAMP DE
ALTER TABLE t ADD COLUMN c INT;
ALTER TABLE t DROP COLUMN c;
ALTER TABLE t RENAME COLUMN a TO b;
ALTER TABLE t RENAME TO t2;
DROP TABLE IF EXISTS t CASCADE;
TRUNCATE TABLE t RESTART IDENTITY;
CREATE INDEX idx_t_name ON t(name);
CREATE UNIQUE INDEX idx_t_name_u ON t(name);
CREATE INDEX idx_t_partial ON t(name) WHERE name IS NOT NULL;
```

11.2 10.2 DML Quick Reference

```
INSERT INTO t (name) VALUES ('a'), ('b');
INSERT INTO t (name) VALUES ('a') ON CONFLICT (name) DO UPDATE SET name = EXCLUDED.name; -
INSERT INTO t (name) VALUES ('a') ON DUPLICATE KEY UPDATE name = VALUES(name); -
UPDATE t SET name = 'x' WHERE id = 1;
DELETE FROM t WHERE id = 1;
INSERT INTO t (name) VALUES ('a') RETURNING id; -- PG/SQLite
```

11.3 10.3 DQL Quick Reference

```
SELECT DISTINCT col FROM t
JOIN t2 ON t.id = t2.t_id
```

WHERE cond **GROUP BY** col **HAVING** agg_cond
ORDER BY col **LIMIT** n **OFFSET** m;

Execution order: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.

11.4 10.4 JOIN Types, One Line Each

Join	One-liner
INNER JOIN	Only rows matching in both tables
LEFT JOIN	All of left + matches from right, NULL otherwise
RIGHT JOIN	All of right + matches from left (rewrite as LEFT)
FULL OUTER JOIN	All rows from both sides, NULL where unmatched
CROSS JOIN	Cartesian product — every row × every row
Self join	Table joined to itself for hierarchies
NATURAL JOIN	Implicit join on same-named columns — avoid
LATERAL JOIN (PG)	Subquery can reference preceding FROM tables

11.5 10.5 Aggregate Functions

COUNT(*), COUNT(col), COUNT(DISTINCT col), SUM(col), AVG(col), MIN(col), MAX(col), STRING_AGG/GROUP_CONCAT, ARRAY_AGG (PG), JSON_AGG (PG), BOOL_AND/BOOL_OR (PG).

11.6 10.6 Window Functions

ROW_NUMBER(), RANK(), DENSE_RANK(), NTILE(n), PERCENT_RANK(), LAG(), LEAD(), FIRST_VALUE(), LAST_VALUE(), SUM()/AVG()/MIN()/MAX() OVER (...), CUME_DIST(), PERCENTILE_CONT().

11.7 10.7 Date Functions One-Liners

Task	PostgreSQL	MySQL	SQLite
Now	NOW()	NOW()	datetime('now')
Year	EXTRACT(YEAR FROM d)	YEAR(d)	strftime('%Y',d)
Add days	d + INTERVAL '7 days'	DATE_ADD(d, INTERVAL 7 DAY)	date(d,'+7 days')
Diff	d1 - d2	DATEDIFF(d1,d2)	julianday(d1) - julianday(d2)

11.8 10.8 String Functions One-Liners

\\|\\|/CONCAT(), LENGTH(), UPPER()/LOWER(), TRIM(), SUBSTR()/SUBSTRING(), POSITION()/INSTR(), REPLACE(), SPLIT_PART()/SUBSTRING_INDEX().

11.9 10.9 JSON One-Liners (PostgreSQL)

-> (get JSON), ->> (get text), @> (contains), ? (key exists), jsonb_build_object(), jsonb_array_elements().

11.10 10.10 Top 15 Dialect Gotchas

1. NOT IN with a NULL-containing subquery returns zero rows — use NOT EXISTS.
2. LEFT JOIN + filter on the right table in WHERE silently becomes an INNER JOIN — filter in ON instead.
3. LIKE '%foo' (leading wildcard) can't use a B-tree index.
4. MySQL has no RETURNING — use LAST_INSERT_ID().
5. MySQL has no FULL OUTER JOIN — emulate with UNION of LEFT/RIGHT joins.
6. SQLite can't ALTER COLUMN type — recreate the table.
7. CHAR pads with spaces; VARCHAR doesn't.
8. Never use FLOAT/REAL for money — use NUMERIC/DECIMAL.
9. PostgreSQL TIMESTAMP (no TZ) vs TIMESTAMPTZ — default to the latter.
10. LAST_VALUE() with the default window frame usually returns the wrong thing — widen the frame explicitly.
11. COUNT(1) is not faster than COUNT(*) — that's a myth.
12. Division by zero errors in PostgreSQL but returns NULL in MySQL.
13. MySQL LIKE is case-insensitive by default; PostgreSQL LIKE is case-sensitive (use ILIKE).
14. SQLite has no native JSON/ARRAY/ENUM types — emulate with TEXT/CHECK.
15. Deep OFFSET pagination re-scans and discards every prior row — use keyset pagination.

11.11 10.11 Top 10 Performance Tips

1. Index every foreign key column (auto-indexed in MySQL InnoDB, not in PostgreSQL/SQLite).
2. Avoid SELECT * — it blocks index-only scans and breaks on schema changes.
3. Put the leading column of a composite index on the most commonly filtered field.
4. Use EXPLAIN ANALYZE before assuming why a query is slow.
5. Prefer EXISTS over IN for large subqueries.
6. Avoid wrapping indexed columns in functions unless you have a matching functional index.
7. Use LIMIT with an indexed ORDER BY for top-N queries.
8. Batch INSERT/UPDATE operations instead of looping row by row.
9. Watch for accidental Cartesian products from a missing join condition.
10. Run ANALYZE/VACUUM ANALYZE after large data changes so the planner's statistics stay accurate.

11.12 10.12 Top 10 Patterns as One-Liners

-- Top N per group

```
SELECT * FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY cat ORDER BY val DESC) rn FROM t)
```

-- Running total

```
SELECT d, SUM(v) OVER (ORDER BY d) FROM t;
```

-- Nth highest

```
SELECT v FROM (SELECT v, DENSE_RANK() OVER (ORDER BY v DESC) r FROM t) x WHERE r = 3;
```

-- Duplicates

```
SELECT k, COUNT(*) FROM t GROUP BY k HAVING COUNT(*) > 1;
```

-- Keyset pagination

```
SELECT * FROM t WHERE id > :last_id ORDER BY id LIMIT 20;
```

-- Upsert (PG)

```
INSERT INTO t (k, v) VALUES (1, 'x') ON CONFLICT (k) DO UPDATE SET v = EXCLUDED.v;
```

-- Recursive hierarchy

```
WITH RECURSIVE cte AS (SELECT * FROM t WHERE parent_id IS NULL UNION ALL SELECT t.* FROM t
```

-- Pivot

```
SELECT k, SUM(CASE WHEN cat='a' THEN v END), SUM(CASE WHEN cat='b' THEN v END) FROM t GROUP
```

-- Percent of total

```
SELECT v, v * 100.0 / SUM(v) OVER () FROM t;
```

```
-- Latest row per group (PG)
SELECT DISTINCT ON (k) * FROM t ORDER BY k, d DESC;
```

DOCUMENT COMPLETE. Parts 1–10 covering SQL Fundamentals, DDL, DML, DQL, Advanced SQL, Transactions & Concurrency, Performance & Optimization, Dialect Differences, Interview Preparation, and the Cheat Sheet — built as self-contained, RAG-chunk-friendly ### subsections against one canonical six-table schema (users, products, orders, order_items, employees, departments).

Note: Entries tagged [verify] cite a specific version number or feature-availability claim (e.g. “MySQL 8.0+”, “SQLite 3.35+”) — please spot-check these against current dialect documentation before publishing, since minor-version support can shift.